

Matplotlib

July 12, 2026

PAO25_25_09_Python_Visualization

July 10, 2026

01MIAR - Representación Gráfica

Kevin Xavier Suasnavas Villarreal

Docente: Elvis Pachacama

Repositorio GitHub

Link a mi GitHub: <https://github.com/kevinsuas433/Machine2026.git>

1 Representación gráfica

- **Matplotlib básico**
 - plot
 - legend
 - decoradores
 - anotaciones
 - guardar figuras
- **Representación gráfica con Pandas**
 - gráficos de líneas
 - barras
 - histogramas
 - pastel
 - áreas
- **Introducción a Seaborn**

<https://seaborn.pydata.org/tutorial.html>

2 Referencias

- <https://plotdb.com/>
- <https://plotnine.org/>
- <https://d3js.org/>
- <https://plotly.com/python/>

- <https://bokeh.org/>
 - <https://shiny.posit.co/>
 - <https://www.tableau.com/>
-

3 1. Matplotlib

- Librería estándar para realizar gráficos en Python.
- Permite crear visualizaciones altamente personalizables.
- Es una librería de bajo nivel con gran control sobre los gráficos.

3.1 1.0.1 Elementos de visualización

En **Matplotlib**, una visualización está compuesta por diferentes elementos que permiten organizar y representar la información de forma clara.

- **Gráfico:** Es la representación visual de un conjunto de datos. Existen diferentes tipos de gráficos como líneas, barras, pastel, dispersión, histogramas, entre otros.
 - **Subfigura (Axes):** Es el área donde se dibuja un gráfico dentro de una figura. Una figura puede contener una o varias subfiguras.
 - **Figura (Figure):** Es el contenedor principal que agrupa uno o varios gráficos. Además, puede incluir un título general, leyendas, etiquetas y otras configuraciones.
-

Para trabajar con gráficos utilizaremos principalmente las siguientes librerías:

- **Matplotlib:** Biblioteca principal para crear gráficos.
- **NumPy:** Permite generar y manipular datos numéricos.
- **Pandas:** Facilita el manejo de tablas y conjuntos de datos.

Las funciones más utilizadas son:

- `plot(data)` → Crea un gráfico utilizando los datos proporcionados.
- `show()` → Muestra en pantalla todos los gráficos generados.

```
[2]: # =====
# Importación de las librerías necesarias para la visualización
# =====

# Matplotlib se utiliza para crear gráficos.
import matplotlib.pyplot as plt

# NumPy permite generar y manipular arreglos numéricos.
import numpy as np

# Pandas facilita el manejo de tablas de datos.
import pandas as pd
```

```
[3]: # =====
# Mostrar los diferentes motores (Backends) disponibles
# para representar gráficos en Matplotlib.
#
# En Jupyter normalmente se utiliza "inline", ya que permite
# visualizar las figuras directamente dentro del Notebook.
# =====

%matplotlib --list
```

Available matplotlib backends: ['agg', 'auto', 'cairo', 'gtk3', 'gtk3agg', 'gtk3cairo', 'gtk4', 'gtk4agg', 'gtk4cairo', 'inline', 'macosx', 'nbagg', 'notebook', 'osx', 'pdf', 'pgf', 'ps', 'qt', 'qt5', 'qt5agg', 'qt5cairo', 'qt6', 'qtagg', 'qtcairo', 'svg', 'template', 'tk', 'tkagg', 'tkcairo', 'webagg', 'wx', 'wxagg', 'wxcairo']

```
[4]: # =====
# Activar el backend "inline".
#
# Esto permite que todos los gráficos aparezcan directamente
# debajo de la celda donde se ejecuta el código.
# =====

%matplotlib inline
```

```
[5]: # =====
# Generar una matriz aleatoria y representarla mediante
# un gráfico de líneas.
# =====

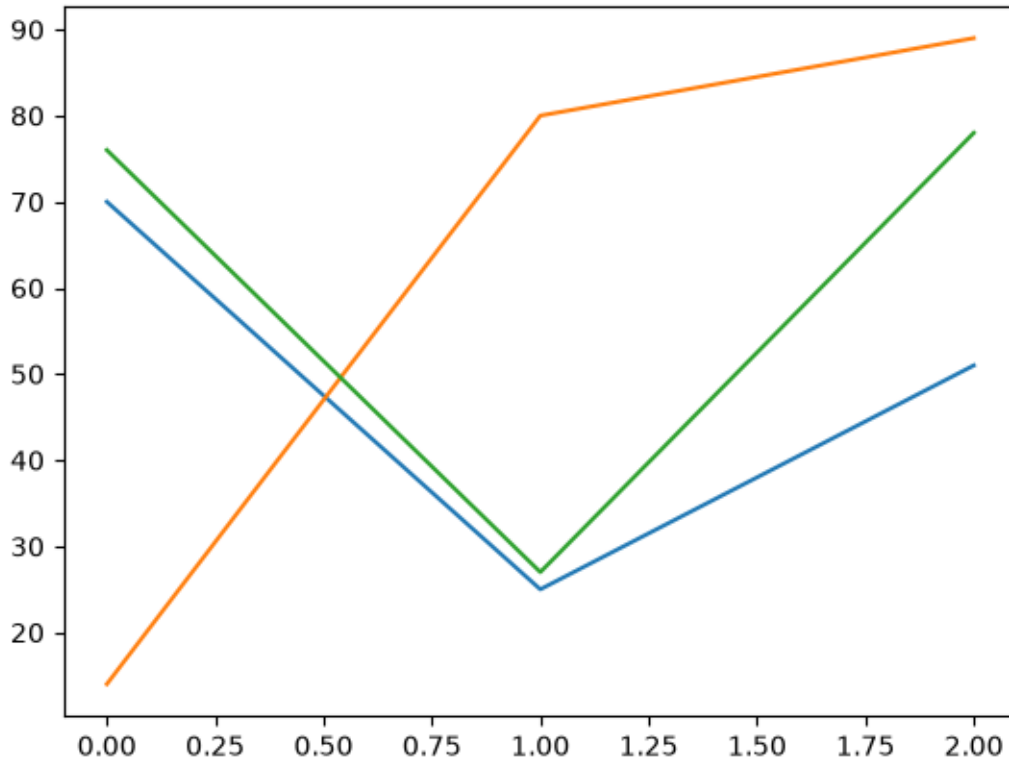
# Generar una matriz de 3 filas por 3 columnas con números
# aleatorios enteros entre 0 y 99.
data = np.random.randint(100, size=(3, 3))

# Mostrar la matriz generada.
display(data)

# Crear el gráfico utilizando los datos generados.
plt.plot(data)

# Mostrar el gráfico.
plt.show()
```

```
array([[70, 14, 76],
       [25, 80, 27],
       [51, 89, 78]], dtype=int32)
```



3.1.1 Modo interactivo y representación de datos

Matplotlib permite trabajar en diferentes modos de visualización.

- **%matplotlib notebook:** activa el modo interactivo, permitiendo hacer zoom, mover y explorar el gráfico directamente en Jupyter Notebook.
- **%matplotlib inline:** muestra los gráficos como imágenes estáticas dentro del Notebook. Es el modo más utilizado para la elaboración de informes.

En un gráfico también es posible especificar de forma independiente los valores del eje **X** y del eje **Y**. Para ello, ambos conjuntos de datos deben tener la misma cantidad de elementos.

```
[6]: # =====
# Activar el modo interactivo de Matplotlib.
# Este modo permite interactuar con los gráficos (zoom, mover,
# guardar, etc.) directamente desde Jupyter Notebook.
# =====

%matplotlib notebook

# Desactivar el modo interactivo para evitar que los gráficos
# anteriores se modifiquen al crear nuevas figuras.
plt.ioff()
```

```
# Volver al modo "inline", que muestra los gráficos como
# imágenes estáticas dentro del Notebook.
%matplotlib inline
```

```
[7]: # =====
# Crear un gráfico especificando por separado los valores
# del eje X y del eje Y.
# =====

# Generar los valores del eje X desde 75 hasta 99.
x_data = np.arange(75, 100)

# Generar 25 valores aleatorios entre 0 y 79 para el eje Y.
y_data = np.random.randint(80, size=25)

# Mostrar los datos generados para verificar su contenido.
print("Valores del eje X:")
print(x_data)

print("\nValores del eje Y:")
print(y_data)

# Verificar que ambos arreglos tengan la misma cantidad
# de elementos antes de crear el gráfico.
if x_data.size == y_data.size:

    # Crear el gráfico de líneas.
    plt.plot(x_data, y_data)

    # Agregar un título descriptivo.
    plt.title("Gráfico de líneas")

    # Etiquetar los ejes.
    plt.xlabel("Eje X")
    plt.ylabel("Eje Y")

    # Mostrar el gráfico.
    plt.show()

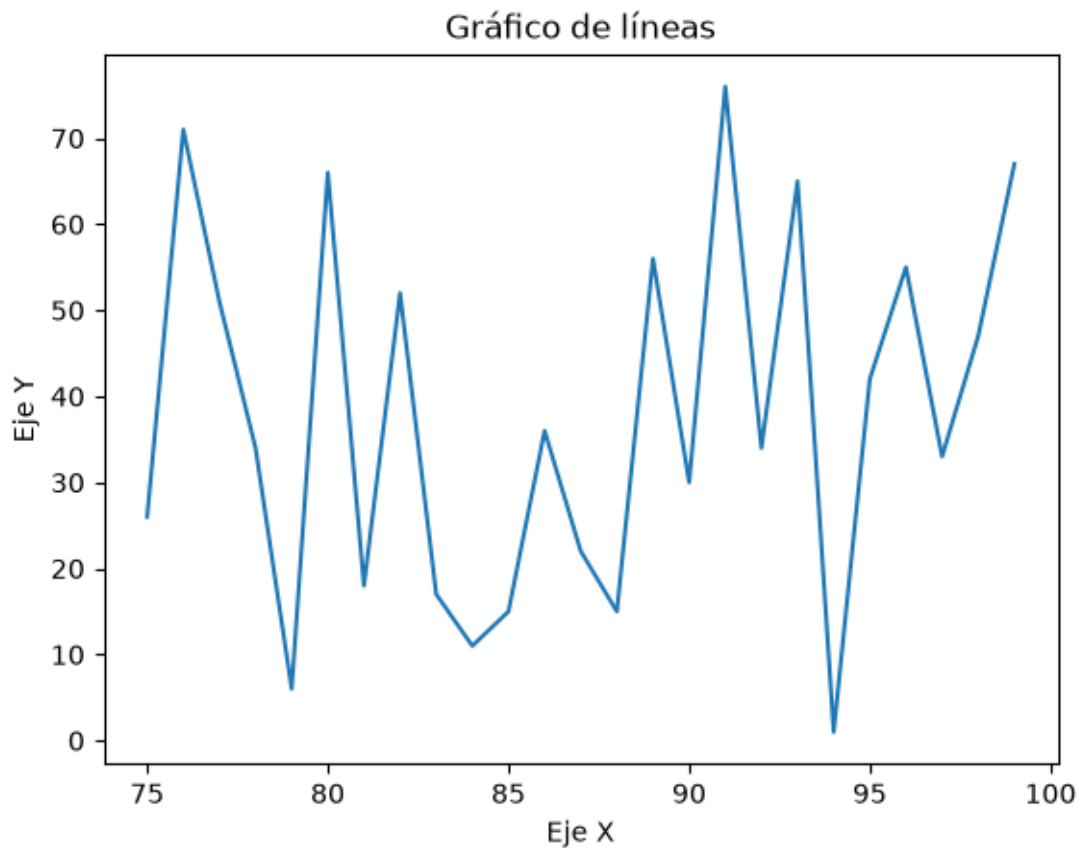
else:
    # Informar si las dimensiones son diferentes.
    print(f"Error: X tiene {x_data.size} elementos y Y tiene {y_data.size}.")
```

Valores del eje X:

```
[75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98
 99]
```

Valores del eje Y:

```
[26 71 51 34 6 66 18 52 17 11 15 36 22 15 56 30 76 34 65 1 42 55 33 47 67]
```



3.2 1.1 Figure y Axes

En Matplotlib, los gráficos se organizan mediante dos elementos principales:

- **Figure:** Es el contenedor principal donde se dibujan todos los gráficos. Una figura puede contener uno o varios gráficos.
- **Axes:** Son las áreas individuales donde se representa cada gráfico. Cada *Axes* corresponde a un subplot dentro de una figura.

La función `plt.subplots()` permite crear de forma simultánea una figura y los subgráficos que contendrá.

```
[8]: # =====  
# Crear una figura con una matriz de subgráficos.  
# En este ejemplo se crean 3 filas y 2 columnas de subplots.  
# =====
```

```
# Crear una figura con 3 filas y 2 columnas de gráficos.
fig, axes = plt.subplots(3, 2)

# Mostrar el tipo de dato de la variable axes.
print("Tipo de la variable axes:")
print(type(axes))

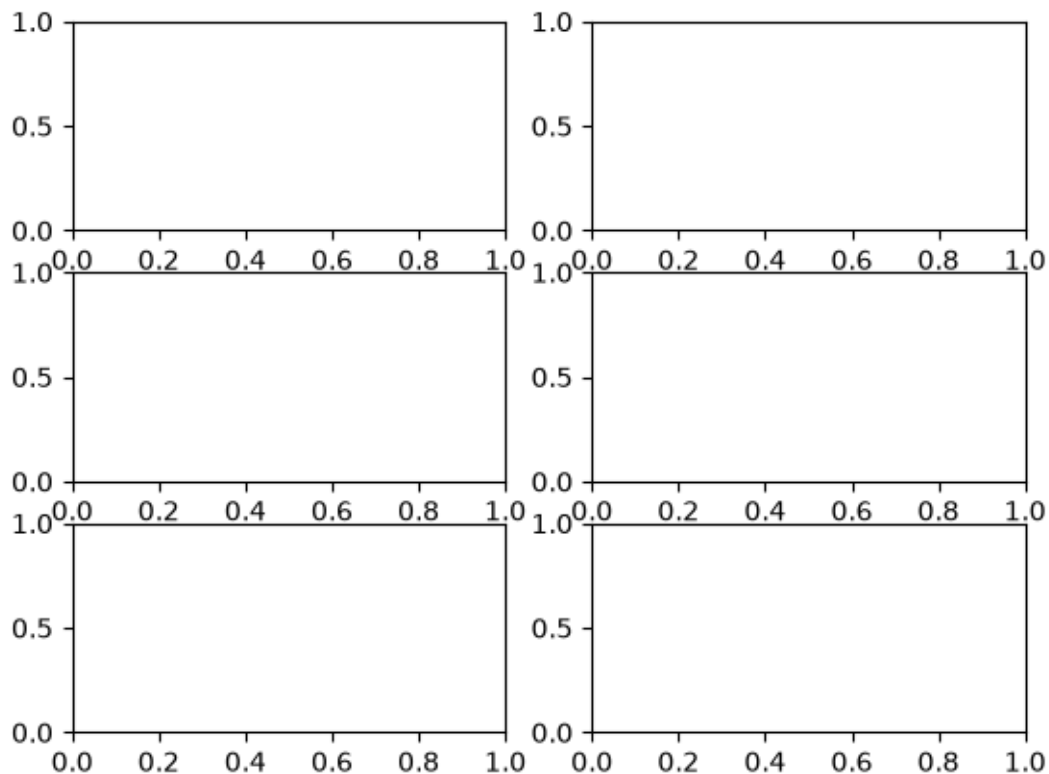
# Mostrar la forma (filas, columnas) de la matriz de subgráficos.
print("\nDimensiones de la matriz de subplots:")
print(axes.shape)
```

Tipo de la variable axes:

<class 'numpy.ndarray'>

Dimensiones de la matriz de subplots:

(3, 2)



```
[9]: # =====
# Generar los datos que se utilizarán en los gráficos.
# =====

# Datos para el primer gráfico.
```

```

x_data1 = np.arange(0, 10)
y_data1 = np.random.randint(100, size=10)

# Datos para el segundo gráfico.
x_data2 = np.arange(-10, 0)
y_data2 = np.random.randint(1000, size=10)

```

```

[10]: # =====
# Crear una figura con dos subgráficos y representar
# los datos generados anteriormente.
# =====

# Crear una figura con 1 fila y 2 columnas.
fig, axes = plt.subplots(1, 2)

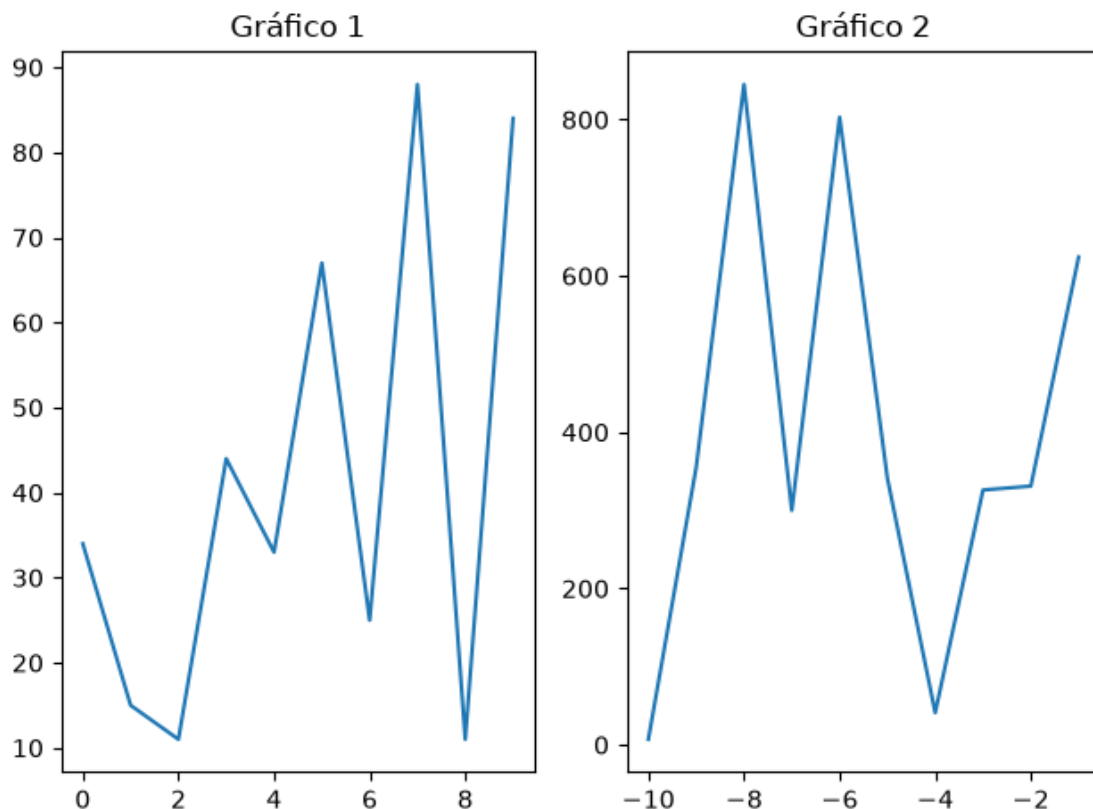
# -----
# Primer subplot
# -----
axes[0].plot(x_data1, y_data1)
axes[0].set_title("Gráfico 1")

# -----
# Segundo subplot
# -----
axes[1].plot(x_data2, y_data2)
axes[1].set_title("Gráfico 2")

# Ajustar automáticamente los espacios entre los gráficos.
plt.tight_layout()

# Mostrar ambos gráficos.
# Se recomienda utilizar plt.show() en lugar de fig.show().
plt.show()

```

Nota:

La función `plt.subplots(filas, columnas)` devuelve dos objetos:

- **fig**: representa la figura completa.
- **axes**: contiene todos los subgráficos creados.

Si se crean varios subgráficos, **axes** será un arreglo de NumPy que permite acceder a cada gráfico mediante índices, por ejemplo:

- `axes[0]`
- `axes[1]`
- `axes[0,1]` (cuando existen varias filas y columnas).

3.2.1 Compartir el eje X y personalizar los ejes

En algunos casos es útil representar dos conjuntos de datos que comparten el mismo eje **X**, pero utilizan escalas diferentes en el eje **Y**.

Para ello, Matplotlib proporciona el método `twinx()`, que crea un segundo eje vertical asociado al mismo eje horizontal.

Además, tanto la figura como los subgráficos permiten personalizar diferentes características, como:

- Límites de los ejes.
- Etiquetas de los ejes.

- Títulos.
- Escalas.
- Estilos de visualización.

```
[11]: # =====
# Crear un segundo eje Y compartiendo el mismo eje X.
# =====

# Crear una figura con un único subplot.
fig, ax = plt.subplots(1, 1)

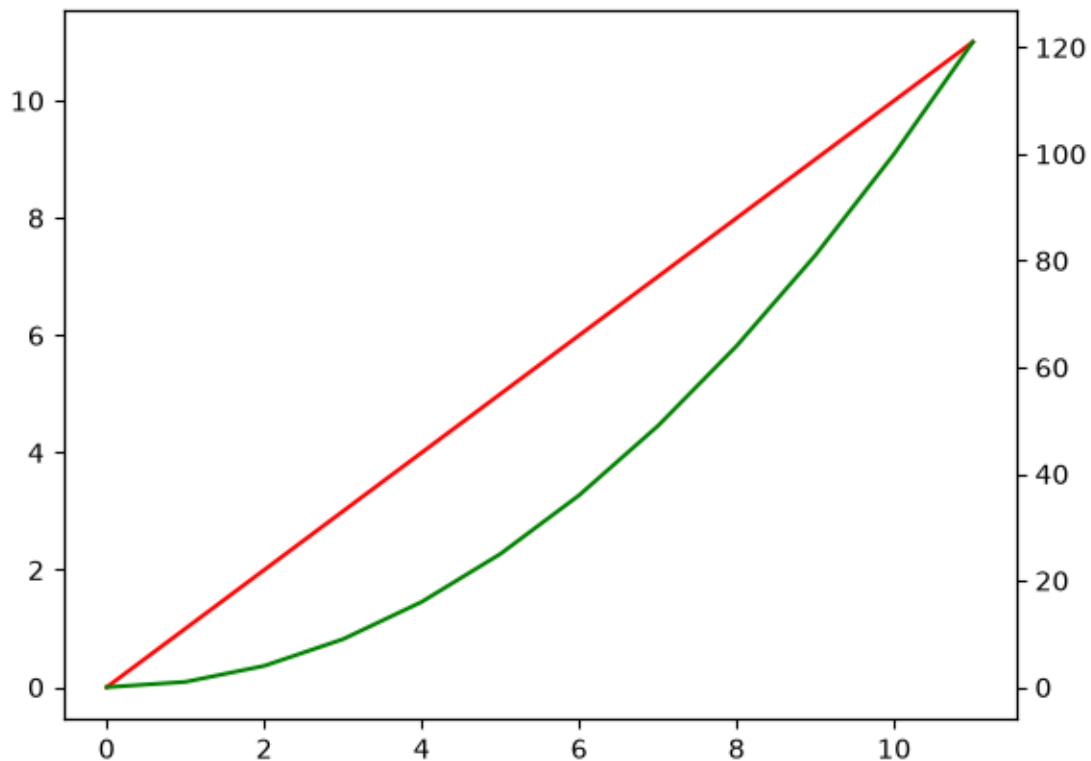
# Generar valores desde 0 hasta 11.
x = np.arange(12)

# Dibujar la primera función en color rojo.
ax.plot(x, "r")

# Crear un segundo eje Y que comparte el mismo eje X.
ax2 = ax.twinx()

# Dibujar una segunda función utilizando el nuevo eje Y.
ax2.plot(x ** 2, "g")

# Mostrar la figura.
plt.show()
```



```

[12]: # =====
# Crear nuevamente dos subgráficos y personalizar
# la configuración de cada eje.
# =====

# Crear una figura con dos subgráficos.
fig, axes = plt.subplots(1, 2)

# -----
# Primer gráfico
# -----
axes[0].plot(x_data1, y_data1)

# Configurar el rango del eje Y.
axes[0].set_ylim([0, 1000])

# Configurar el rango del eje X.
axes[0].set_xlim([-1, 12])

# Etiqueta del eje X.
axes[0].set_xlabel("Years")

# Etiqueta del eje Y.
axes[0].set_ylabel("Dollars")

# -----
# Segundo gráfico
# -----
axes[1].plot(x_data2, y_data2)

# Configurar el rango del eje Y.
axes[1].set_ylim([0, 1000])

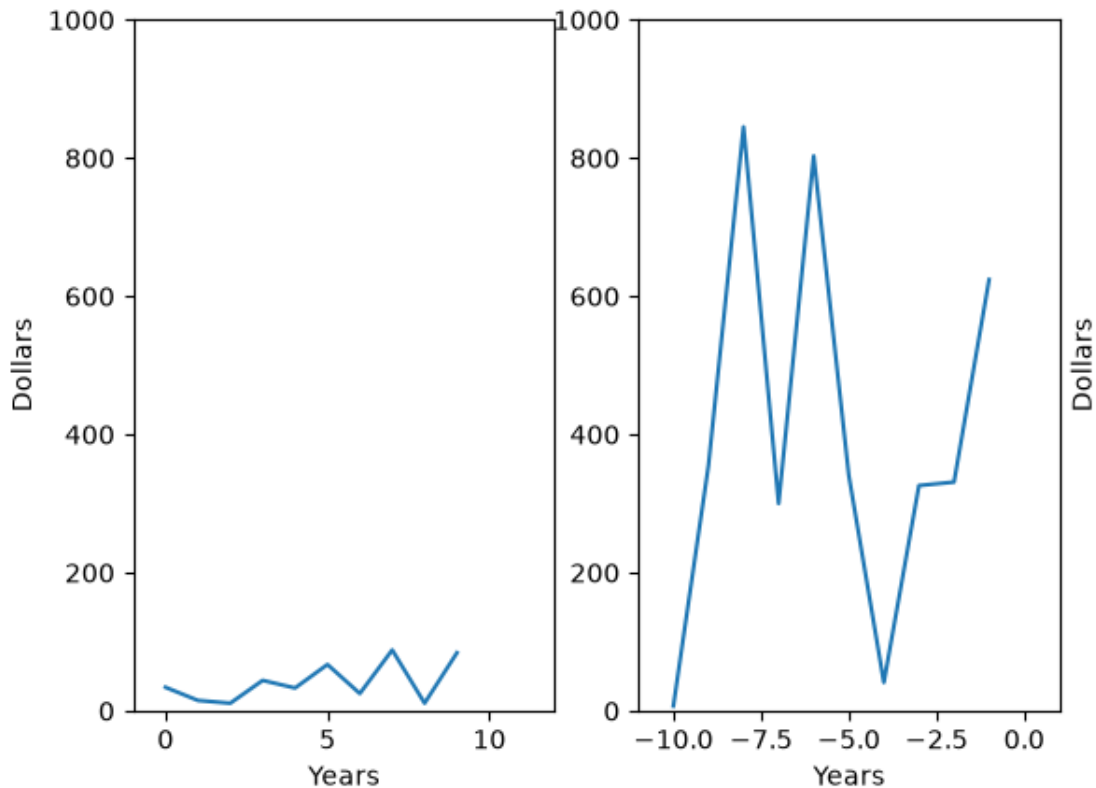
# Configurar el rango del eje X.
axes[1].set_xlim([-11, 1])

# Etiqueta del eje X.
axes[1].set_xlabel("Years")

# Etiqueta del eje Y.
axes[1].set_ylabel("Dollars", loc="center", labelpad=-210)

# Mostrar ambos gráficos.
plt.show()

```



¿Qué hacen estos métodos?

- `set_xlim()`: establece el rango que se mostrará en el eje X.
- `set_ylim()`: establece el rango visible del eje Y.
- `set_xlabel()`: asigna un nombre al eje horizontal.
- `set_ylabel()`: asigna un nombre al eje vertical.
- `twinx()`: crea un segundo eje Y compartiendo el mismo eje X.

3.2.2 1.1.1 Más métodos

Método `plot()` El método `plot()` es una de las funciones principales de Matplotlib y permite representar datos mediante gráficos de líneas.

Su sintaxis general es la siguiente:

```
plt.plot(x, y, style, label, linewidth=1, alpha=1)
```

Los parámetros más utilizados son:

- **x**: valores del eje horizontal.
- **y**: valores del eje vertical.
- **style**: define el color, marcador y tipo de línea utilizando el formato `[color][marker][line]`.
- **label**: nombre que aparecerá en la leyenda del gráfico.
- **linewidth**: grosor de la línea.

- **alpha**: nivel de transparencia de la línea, donde 0 es completamente transparente y 1 es completamente opaca.

La documentación oficial de Matplotlib puede consultarse en:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

```
[13]: # =====
# Ejemplo del método plot() utilizando varios parámetros.
# =====

# Generar 10 valores aleatorios entre 0 y 99 para el eje Y.
y_data = np.random.randint(100, size=10)

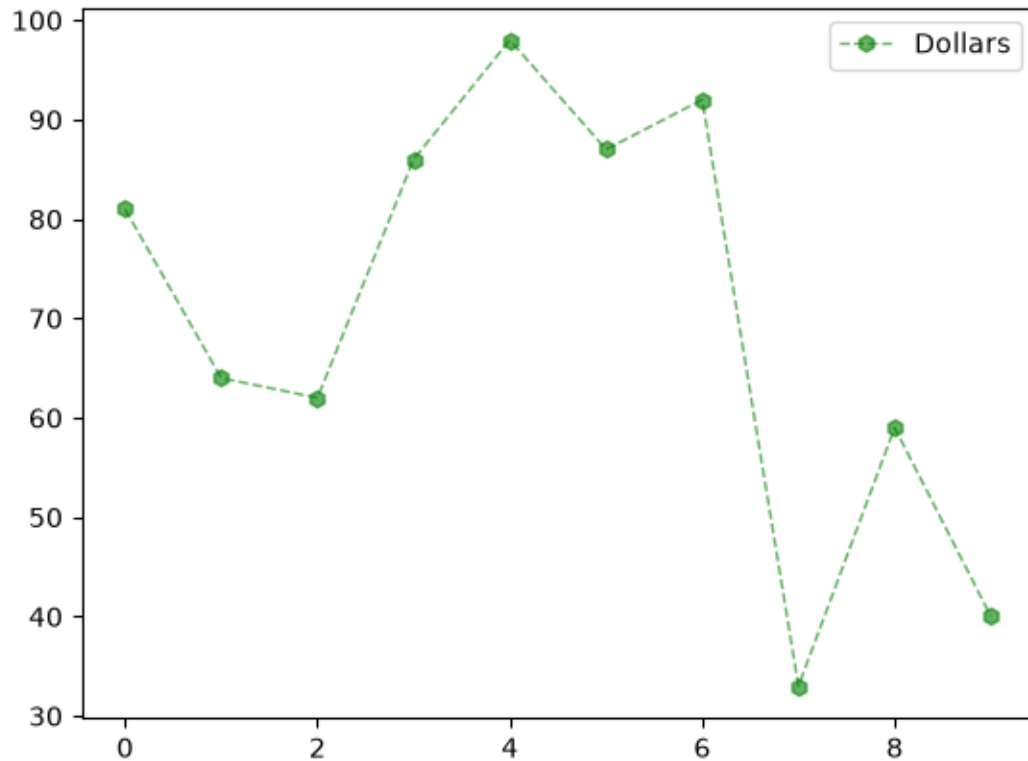
# Crear los valores del eje X desde 0 hasta 9.
x_data = np.arange(10)

# Crear una figura con un único gráfico.
fig, axis = plt.subplots(1, 1)

# Dibujar el gráfico especificando diferentes parámetros:
# g -> color verde (green)
# h -> marcador hexagonal
# -- -> línea discontinua
axis.plot(
    x_data,
    y_data,
    "gh--",
    label="Dollars",
    linewidth=1,
    alpha=0.6
)

# Mostrar la leyenda en la mejor posición automáticamente.
axis.legend(loc="best")

# Mostrar el gráfico.
plt.show()
```



3.2.3 Significado del estilo "gh--"

El parámetro `style` combina tres características en una sola cadena de texto:

Código	Significado
g	Color verde (<i>green</i>)
h	Marcador hexagonal
-	Línea discontinua

Por ejemplo:

- "ro-" → línea roja con marcadores circulares.
- "bs:" → línea azul con marcadores cuadrados y línea punteada.
- "k^-" → línea negra con marcadores triangulares.

3.3 Colores disponibles en `plot()`

Código	Color
b	Blue (Azul)
g	Green (Verde)
r	Red (Rojo)

Código	Color
c	Cyan
m	Magenta
y	Yellow (Amarillo)
k	Black (Negro)
w	White (Blanco)

3.4 Marcadores disponibles

Código	Marcador
.	Point marker
,	Pixel marker
o	Circle marker
v	Triangle down
^	Triangle up
<	Triangle left
>	Triangle right
1	Tri down
2	Tri up
3	Tri left
4	Tri right
s	Square
p	Pentagon
*	Star
h	Hexagon 1
H	Hexagon 2
+	Plus
x	X marker
D	Diamond
d	Thin diamond
-	Horizontal line

3.5 Tipos de línea

Código	Descripción
-	Línea continua
--	Línea discontinua
-.	Línea punto-guion
:	Línea punteada

3.6 2.0.1 Representar más de un gráfico en una subfigura

Matplotlib permite representar varias series de datos dentro del mismo gráfico utilizando múltiples llamadas al método `plot()`. Cada serie puede tener un color, marcador y estilo de línea diferente, además de una leyenda que facilite su identificación.

```
[14]: # =====  
# Representar varias series de datos en un mismo gráfico.  
# Cada llamada a plot() agrega una nueva línea al mismo eje.  
# =====  
  
# Crear los valores del eje X desde 0 hasta 9.  
x_data = np.arange(10)  
  
# Crear una figura con un único subplot.  
fig, axis = plt.subplots(1, 1)  
  
# -----  
# Primera serie de datos  
# -----  
# Color: verde  
# Marcador: círculo  
# Línea: continua  
axis.plot(  
    x_data,  
    np.random.randint(100, size=10),  
    "go-",  
    label="Green",  
    linewidth=1,  
    alpha=0.2  
)  
  
# -----  
# Segunda serie de datos  
# -----  
# Color: azul  
# Marcador: x  
# Línea: punto-guion  
axis.plot(  
    x_data,  
    np.random.randint(100, size=10),  
    "bx-.",  
    label="Blue",  
    linewidth=1  
)  
  
# -----  
# Tercera serie de datos
```



```

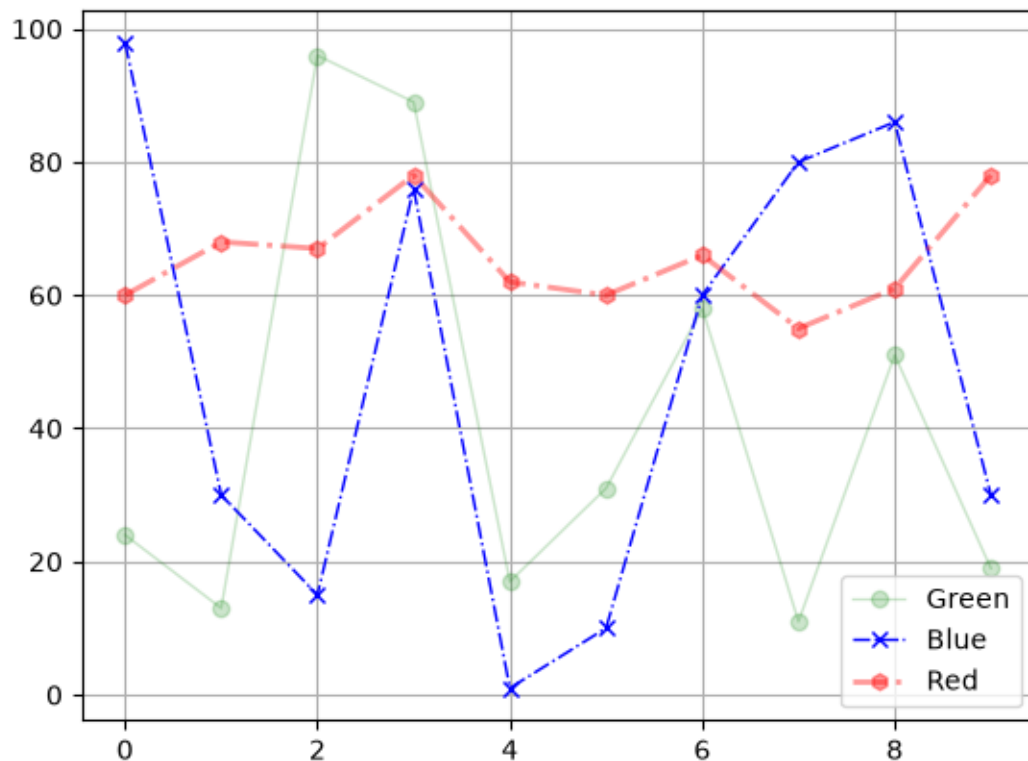
# -----
# Color: rojo
# Marcador: hexágono
# Línea: punto-guion
axis.plot(
    x_data,
    np.random.randint(50, 80, size=10),
    "rh-.",
    label="Red",
    linewidth=2,
    alpha=0.4
)

# Mostrar la leyenda automáticamente en la mejor posición.
axis.legend(loc="best")

# Mostrar una cuadrícula para facilitar la lectura del gráfico.
plt.grid()

# Mostrar el gráfico.
plt.show()

```



Observación

En un mismo gráfico pueden representarse varias series de datos utilizando varias llamadas consecutivas a `plot()`.

Cada serie puede tener:

- Un color diferente.
- Un marcador distinto.
- Un tipo de línea propio.
- Un nivel de transparencia.
- Un nombre para la leyenda mediante el parámetro `label`.

3.7 2.1 Decoradores

Los **decoradores** permiten modificar el aspecto visual de un gráfico para hacerlo más claro y fácil de interpretar.

Algunos de los métodos más utilizados son:

Método	Descripción
<code>axes.set_xticks(lista)</code>	Establece las marcas o etiquetas del eje X.
<code>axes.set_title("Título")</code>	Agrega un título al gráfico.
<code>axes.set_ylabel("Etiqueta")</code>	Asigna una etiqueta al eje Y.
<code>axes.set_frame_on(bool)</code>	Activa o desactiva el marco del gráfico.

La documentación completa puede consultarse en la API oficial de Matplotlib.

3.8 3. Anotaciones

Las **anotaciones** permiten agregar información directamente sobre un gráfico para resaltar un punto o una característica importante.

Los métodos más utilizados son:

Método	Descripción
<code>axes.text(x, y, "texto")</code>	Inserta un texto en una posición determinada.
<code>axes.annotate()</code>	Agrega un texto acompañado de una flecha hacia un punto específico del gráfico.

Las anotaciones son muy útiles para explicar máximos, mínimos, tendencias o cualquier dato relevante dentro de una visualización.

```
[16]: # =====  
# Generar los datos que se utilizarán para el ejemplo.  
# =====
```

```

# Crear 10 valores aleatorios entre 0 y 99.
y_data = np.random.randint(100, size=10)

# Crear los valores del eje X desde 0 hasta 9.
x_data = np.arange(10)

```

```

[17]: # =====
# Crear un gráfico y agregar una anotación.
# =====

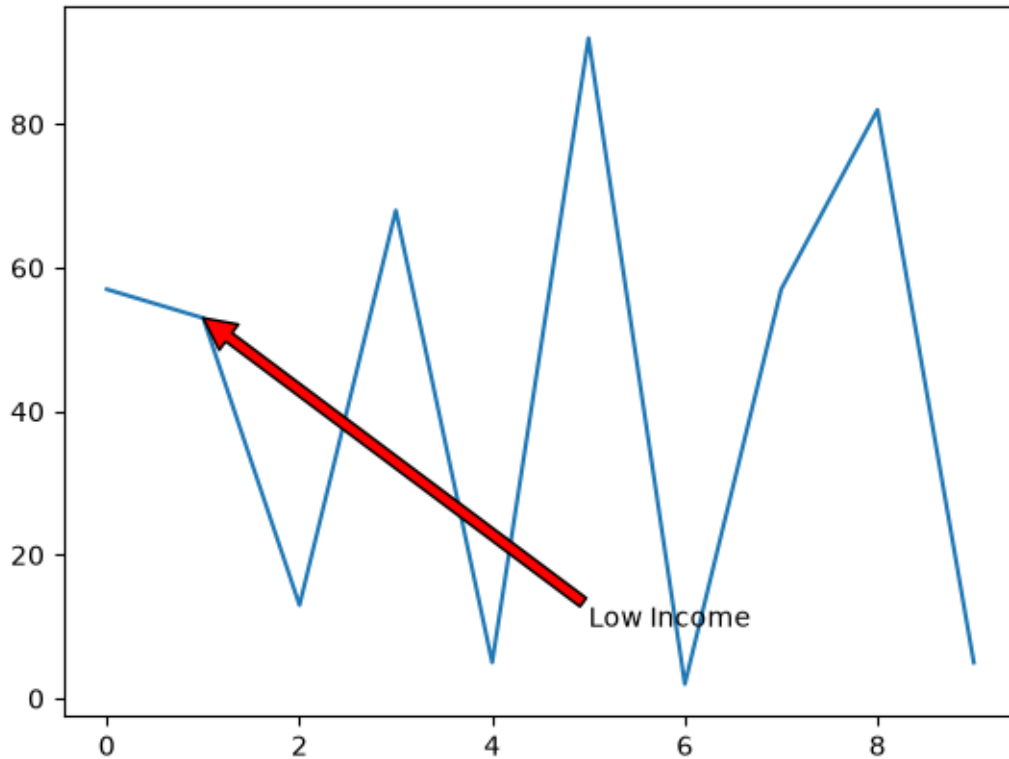
# Crear la figura y el eje.
fig, axis = plt.subplots(1, 1)

# Dibujar el gráfico de líneas.
axis.plot(x_data, y_data)

# -----
# Agregar una anotación.
#
# xy      -> punto que será señalado.
# xytext  -> posición donde aparecerá el texto.
# arrowprops -> configuración de la flecha.
# -----
axis.annotate(
    "Low Income",
    xy=(x_data[1], y_data[1]),
    xytext=(5, 10),
    arrowprops=dict(
        facecolor="red",
        shrink=1.0
    )
)

# Mostrar el gráfico.
plt.show()

```



Nota

En este ejemplo se utiliza `annotate()` porque permite señalar un punto específico mediante una flecha.

También existe el método `text()`, que únicamente coloca un texto sobre el gráfico sin dibujar ninguna flecha.

```
axis.text(x_data[1], y_data[1], "Low Income")
```

Este método resulta útil cuando solo se desea mostrar una etiqueta en una posición determinada.

4 4. Tipos de gráficos

Matplotlib ofrece una gran variedad de gráficos para representar información de forma visual. Cada tipo de gráfico es adecuado según la naturaleza de los datos y el objetivo del análisis.

La documentación oficial de Matplotlib presenta numerosos ejemplos y puede consultarse en:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html

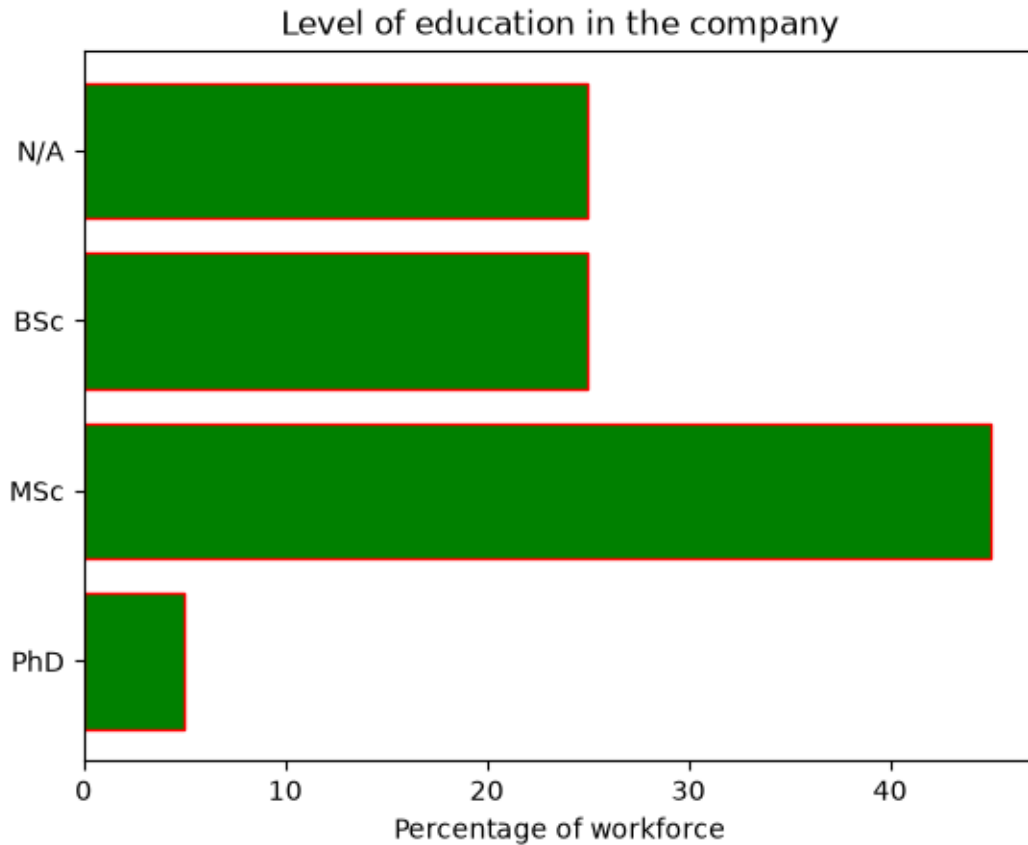
4.0.1 Gráfico de barras horizontales (Horizontal Bar Chart)

El gráfico de barras horizontales (`barh`) permite comparar cantidades entre diferentes categorías.

En este ejemplo se representa el porcentaje de trabajadores según su nivel de educación.

```
[18]: # =====  
# Gráfico de barras horizontales (Horizontal Bar Chart)  
# =====  
  
# Crear una figura con un único gráfico.  
fig, ax = plt.subplots()  
  
# -----  
# Definir los datos que se representarán.  
# -----  
  
# Categorías correspondientes al nivel de educación.  
people = ("PhD", "MSc", "BSc", "N/A")  
  
# Crear las posiciones para cada barra.  
y_pos = np.arange(len(people))  
  
# Porcentaje asociado a cada categoría.  
percentage = [5, 45, 25, 25]  
  
# -----  
# Crear el gráfico de barras horizontales.  
# -----  
ax.barh(  
    y_pos,  
    percentage,  
    align="center",  
    color="green",  
    edgecolor="red"  
)  
  
# -----  
# Configurar las etiquetas del eje Y.  
# -----  
ax.set_yticks(y_pos)  
ax.set_yticklabels(people)  
  
# Si se desea invertir el orden de las categorías,  
# puede utilizarse la siguiente instrucción:  
#  
# ax.invert_yaxis()
```

```
# Etiqueta del eje X.  
ax.set_xlabel("Percentage of workforce")  
  
# Título del gráfico.  
ax.set_title("Level of education in the company")  
  
# Mostrar el gráfico.  
plt.show()
```



¿Qué hace `barh()`?

El método `barh()` crea un gráfico de barras horizontales.

Sus parámetros principales son:

- **y** → posición de cada barra.
- **width** → longitud de cada barra.
- **color** → color de relleno.
- **edgecolor** → color del borde.
- **align** → alineación de las barras.

Este tipo de gráfico es muy útil para comparar categorías cuando sus nombres son

largos, ya que facilita la lectura respecto a un gráfico de barras verticales.

4.0.2 Personalización de un gráfico de barras horizontales

Además de crear un gráfico de barras, Matplotlib permite modificar individualmente el color de cada barra y agregar una leyenda para facilitar la interpretación de los datos.

En este ejemplo se representa la cantidad de kilómetros de carril bici en varias ciudades españolas, utilizando un color diferente para cada una.

```
[19]: # =====  
# Gráfico de barras horizontales con colores personalizados  
# =====  
  
# Cantidad de kilómetros de carril bici por ciudad.  
ls_count = (284487, 244560, 208493, 196764)  
  
# Nombre de las ciudades.  
cities = ("Madrid", "Barcelona", "Sevilla", "Valencia")  
  
# Convertir los valores a miles de kilómetros.  
data = np.array(ls_count) / 1000  
  
# Crear la posición de cada barra.  
y_pos = np.arange(data.size)  
  
# Ancho de las barras.  
width = 0.35  
  
# Nivel de transparencia.  
opacity = 0.7  
  
# Crear la figura y el eje.  
fig, ax = plt.subplots()  
  
# Crear el gráfico de barras horizontales.  
rects1 = ax.barh(  
    y_pos,  
    data,  
    width,  
    alpha=opacity  
)  
  
# -----  
# Cambiar el color de cada barra de manera individual.  
# -----  
rects1[3].set_color("#872f29")  
rects1[2].set_color("#ffcb59")
```

```

rects1[1].set_color("#d6ee39")
rects1[0].set_color("#6ade30")

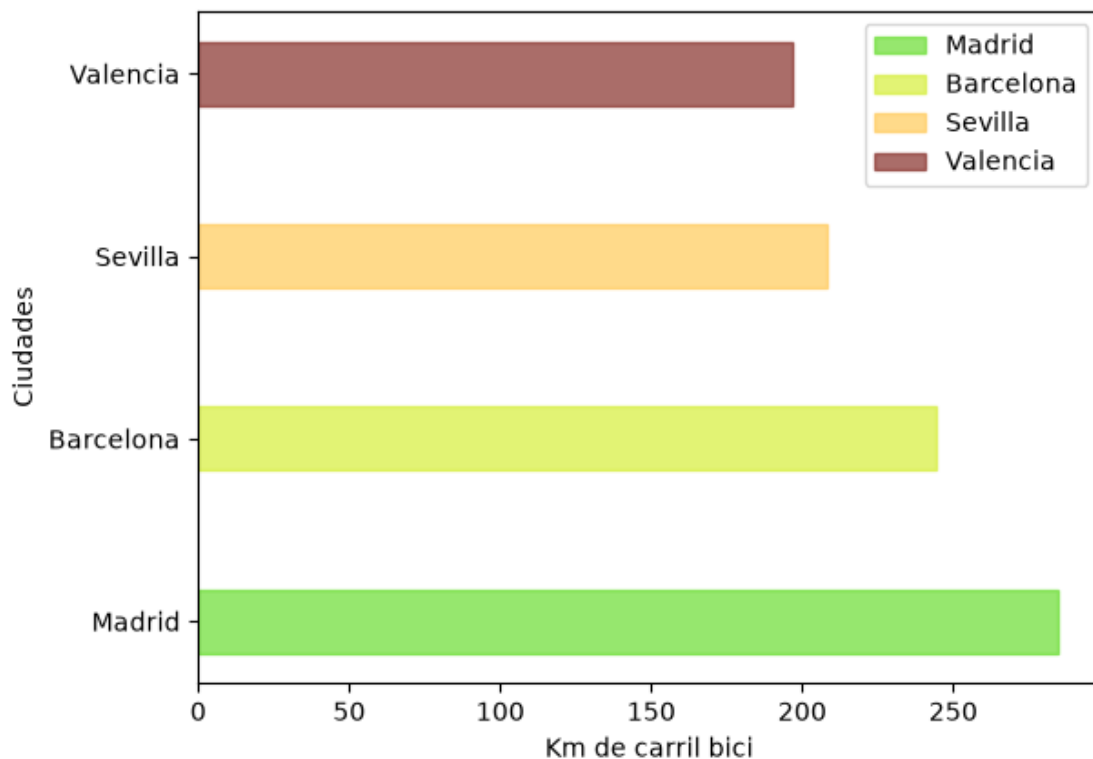
# Configurar las etiquetas del eje Y.
ax.set_yticks(y_pos)
ax.set_yticklabels(cities)

# Etiquetas de los ejes.
ax.set_ylabel("Ciudades")
ax.set_xlabel("Km de carril bici")

# Agregar la leyenda utilizando las barras creadas.
ax.legend(
    (rects1[0], rects1[1], rects1[2], rects1[3]),
    cities
)

# Mostrar el gráfico.
plt.show()

```



¿Qué devuelve `barh()`?

El método `barh()` devuelve una colección de objetos denominada **BarContainer**,

donde cada barra puede modificarse individualmente.

Por ejemplo:

```
rects1[0].set_color("#6ade30")
```

cambia únicamente el color de la primera barra del gráfico.

De esta forma es posible personalizar cada barra con un color, borde o estilo diferente según las necesidades de la visualización.

4.1 Stack Plot (Gráfico de áreas apiladas)

Un **Stack Plot** permite representar cómo varias series de datos contribuyen a un total a lo largo del tiempo.

Cada área corresponde a una categoría y todas se apilan unas sobre otras, facilitando la comparación de su evolución.

```
[20]: # =====  
# Gráfico de áreas apiladas (Stack Plot)  
# =====  
  
# Años que se utilizarán en el eje X.  
x = [2008, 2009, 2010, 2011, 2012]  
  
# Datos correspondientes a cada sistema operativo.  
iOS = [35, 25, 20, 17, 18]  
Android = [0, 15, 35, 45, 60]  
WindowsPhone = [1, 3, 5, 5, 2]  
Others = [64, 57, 40, 33, 20]  
  
# También sería posible agrupar los datos en una sola matriz.  
# y = np.vstack([iOS, Android, WindowsPhone, Others])  
# print(y)  
  
# Etiquetas que aparecerán en la leyenda.  
labels = ["iOS", "Android", "WindowsPhone", "Others"]  
  
# Crear la figura.  
fig, ax = plt.subplots()  
  
# Crear el gráfico de áreas apiladas.  
ax.stackplot(  
    x,  
    iOS,  
    Android,  
    WindowsPhone,  
    Others,  
    labels=labels
```

```

)

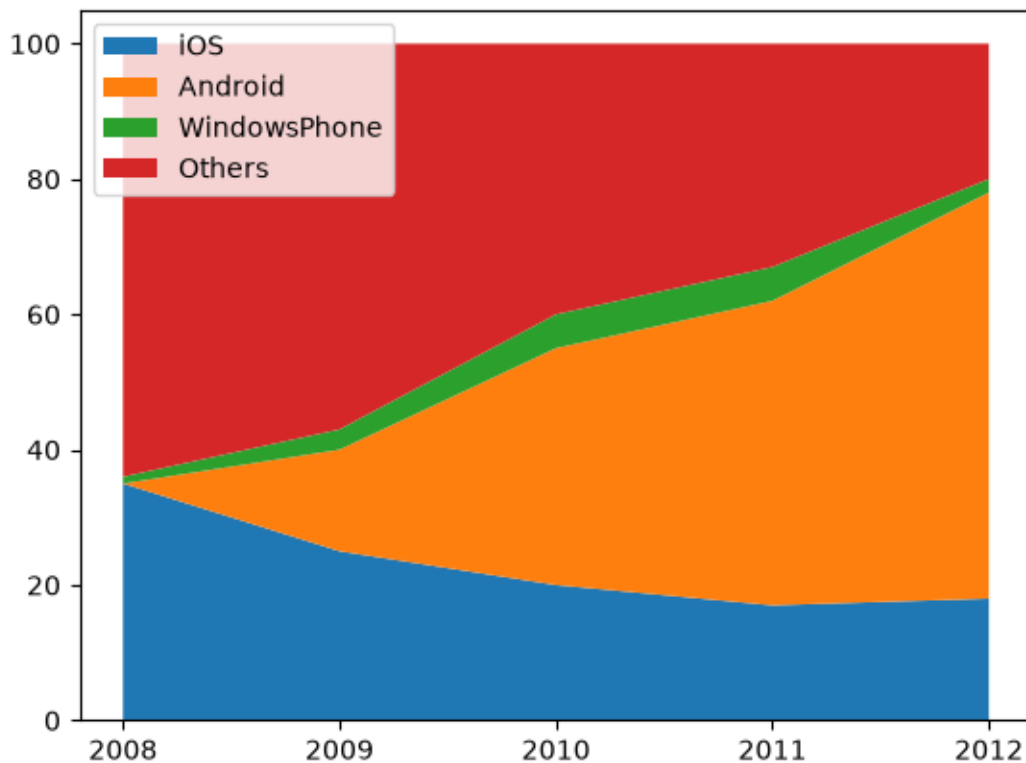
# Otra forma equivalente.
# ax.stackplot(x, y, labels=labels)

# Mostrar únicamente los años definidos.
ax.set_xticks(x)

# Mostrar la leyenda.
ax.legend(loc="upper left")

# Mostrar el gráfico.
plt.show()

```



4.2 Scatter Plot (Gráfico de dispersión)

El gráfico de dispersión permite representar la relación entre dos variables numéricas.

Cada punto corresponde a una observación y puede personalizarse con diferentes colores, tamaños y niveles de transparencia.

```
[21]: # =====
# Gráfico de dispersión (Scatter Plot)
# =====

# Importar la función rand para generar números aleatorios.
from numpy.random import rand

# Crear la figura.
fig, ax = plt.subplots()

# Cantidad de puntos por grupo.
n = 700

# Crear tres grupos de puntos con distintos colores.
for color in ["red", "green", "blue"]:

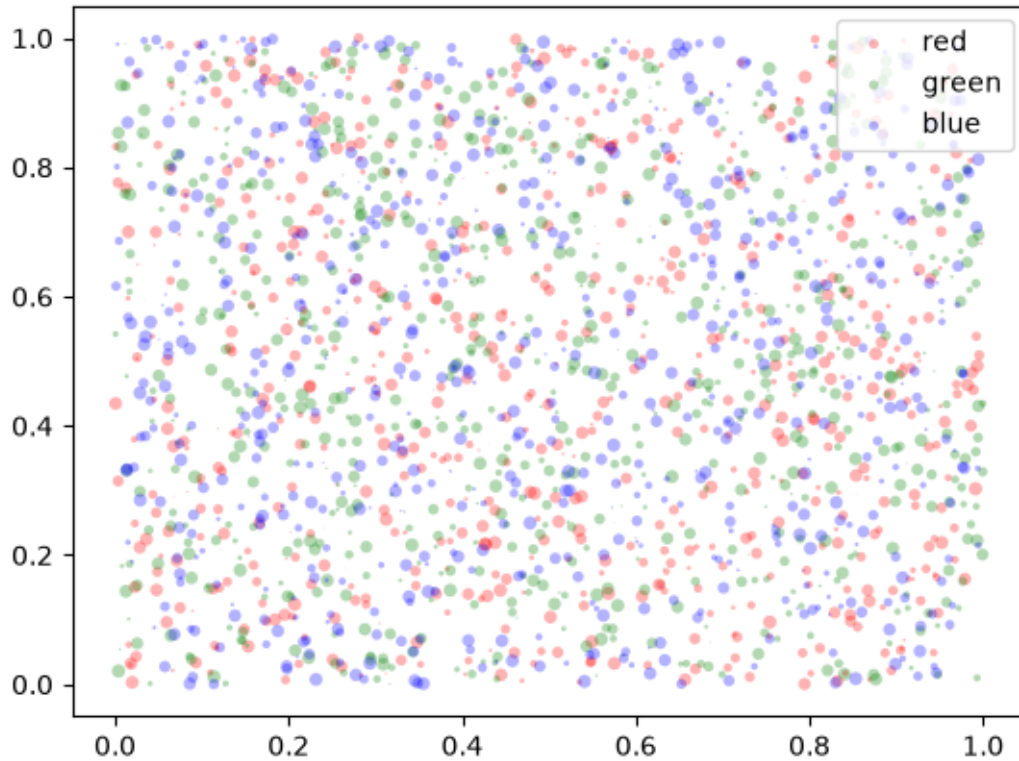
    # Generar coordenadas aleatorias.
    x, y = rand(2, n)

    # Generar tamaños aleatorios para cada punto.
    scale = 25.0 * rand(n)

    # Dibujar el grupo de puntos.
    ax.scatter(
        x,
        y,
        c=color,
        s=scale,
        label=color,
        alpha=0.3,
        edgecolors="none"
    )

# Mostrar la leyenda.
ax.legend()

# Mostrar el gráfico.
plt.show()
```



4.3 4.1 Grabar un gráfico en un archivo

Una vez creado un gráfico, Matplotlib permite almacenarlo en diferentes formatos como:

- PNG
- PDF
- SVG
- EPS

Para ello se utiliza el método:

`fig.savefig()`

Este método guarda la figura exactamente como se visualiza en pantalla.

```
[22]: # =====
# Guardar una figura en un archivo.
# =====

# Importar el módulo para trabajar con rutas.
import os

# Crear la carpeta "res" previamente si no existe.
# En ella se almacenará la imagen generada.
```

```

ruta = os.path.join("res", "o_figure.png")

# Guardar la figura en formato PNG.
# También pueden utilizarse formatos como:
# PDF, SVG o EPS.
fig.savefig(
    ruta,
    format="png"
)

print(f"Figura guardada correctamente en: {ruta}")

```

Figura guardada correctamente en: res\o_figure.png

5 5. Representación gráfica en Pandas

Pandas incorpora una interfaz gráfica que permite crear visualizaciones de forma sencilla utilizando objetos **Series** y **DataFrame**.

Aunque los gráficos se generan mediante métodos propios de Pandas, internamente se utiliza la biblioteca **Matplotlib**, lo que proporciona una gran flexibilidad para personalizar las visualizaciones.

5.0.1 Características

- Permite graficar directamente objetos **Series** y **DataFrame**.
- Utiliza **Matplotlib** como motor de representación.
- Facilita la creación de gráficos con muy pocas líneas de código.

```

[23]: # =====
# Crear un DataFrame y representarlo gráficamente.
# =====

# Generar una matriz aleatoria de 10 filas y 3 columnas.
rand_matrix = np.random.randint(100, size=(10, 3))

# Crear un DataFrame utilizando la matriz generada.
# Las columnas se llamarán: A, X y C.
frame = pd.DataFrame(
    rand_matrix,
    columns=list("AXC")
)

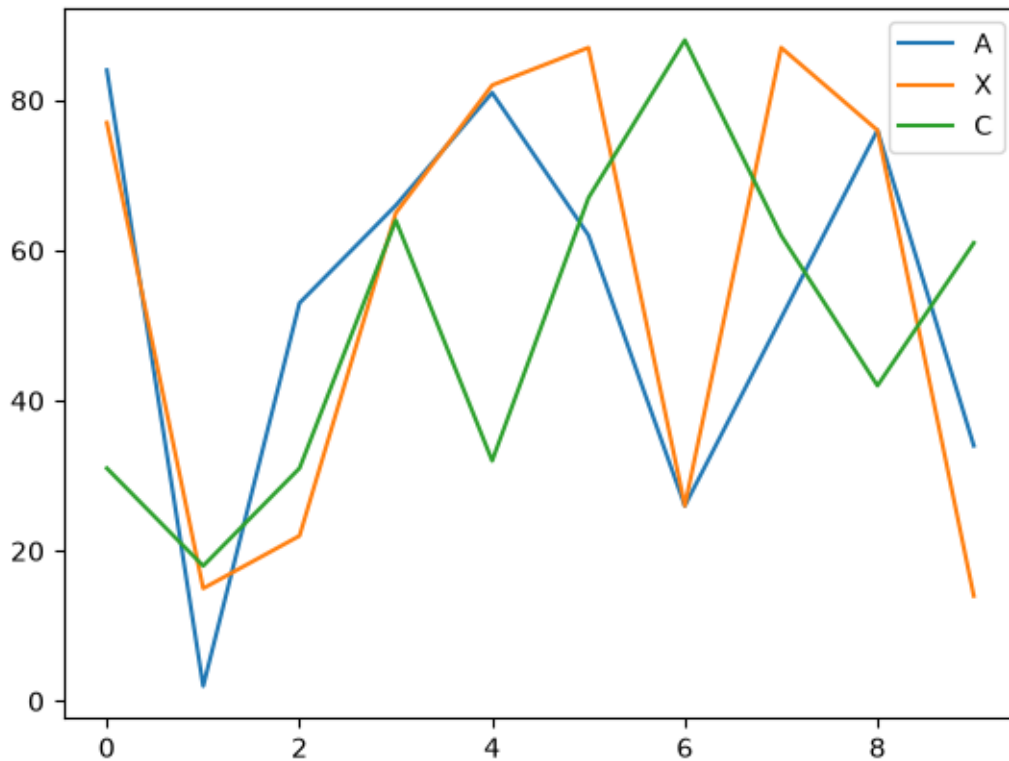
# Crear un gráfico de líneas con los datos del DataFrame.
frame.plot()

# Mostrar el gráfico.
# Aunque Pandas utiliza Matplotlib internamente,

```

```
# todavía es recomendable llamar a plt.show().
plt.show()

# Mostrar el contenido del DataFrame.
display(frame)
```



	A	X	C
0	84	77	31
1	2	15	18
2	53	22	31
3	66	65	64
4	81	82	32
5	62	87	67
6	26	26	88
7	51	87	62
8	76	76	42
9	34	14	61

¿Qué hace `frame.plot()`?

El método `plot()` identifica automáticamente las columnas numéricas del **DataFrame** y genera un gráfico de líneas para cada una de ellas.

De forma predeterminada:

- El índice del DataFrame se utiliza como eje **X**.
- Los valores de las columnas se representan en el eje **Y**.
- Cada columna se dibuja como una serie independiente.
- Se genera automáticamente una leyenda con el nombre de cada columna.

6 6. Método `frame.plot()`

Pandas permite crear gráficos directamente a partir de un **DataFrame** mediante el método `plot()`. Este método utiliza internamente Matplotlib, por lo que es posible personalizar la apariencia del gráfico utilizando diferentes parámetros.

6.1 Sintaxis general

```
frame.plot(
    x=None,
    y=None,
    label=None,
    ax=None,
    style=None,
    kind="line",
    xticks=None,
    yticks=None,
    title=None,
    subplots=False
)
```

6.1.1 Parámetros más utilizados

Parámetro	Descripción
<code>label</code>	Nombre que aparecerá en la leyenda (Series).
<code>ax</code>	Eje donde se dibujará el gráfico.
<code>style</code>	Estilo de la línea.
<code>kind</code>	Tipo de gráfico (<code>line</code> , <code>bar</code> , <code>barh</code> , <code>pie</code> , <code>hist</code> , <code>area</code> , <code>density</code> , <code>kde</code> , etc.).
<code>xticks</code>	Valores del eje X.
<code>yticks</code>	Valores del eje Y.
<code>title</code>	Título del gráfico.
<code>subplots</code>	Crea un gráfico independiente para cada columna del DataFrame.
<code>x, y</code>	Permiten seleccionar qué columnas utilizar en cada eje.

```
[24]: # =====
# Crear un DataFrame y representarlo utilizando subgráficos.
# =====

# Generar una matriz aleatoria de 10 filas y 3 columnas.
```

```

rand_matrix = np.random.randint(100, size=(10, 3))

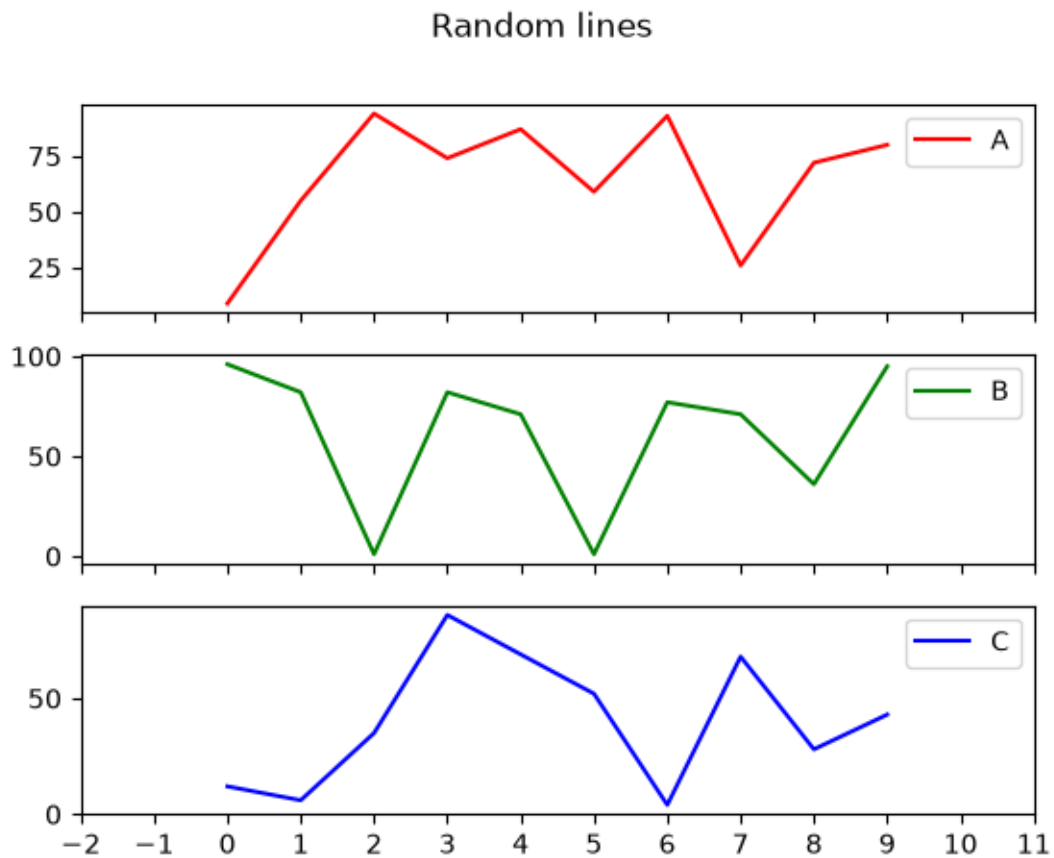
# Crear el DataFrame.
frame = pd.DataFrame(
    rand_matrix,
    columns=list("ABC")
)

# Crear un gráfico para cada columna.
frame.plot(
    style=["r-", "g-", "b-"],
    xticks=range(-2, 12),
    title="Random lines",
    subplots=True
)

# Mostrar el gráfico.
plt.show()

# Mostrar el DataFrame.
display(frame)

```



	A	B	C
0	9	96	12
1	55	82	6
2	94	1	35
3	74	82	86
4	87	71	69
5	59	1	52
6	93	77	4
7	26	71	68
8	72	36	28
9	80	95	43

```
[25]: # =====
# Combinar gráficos de Pandas con Matplotlib.
# =====

# Generar una nueva matriz aleatoria.
rand_matrix = np.random.randint(100, size=(10, 3))

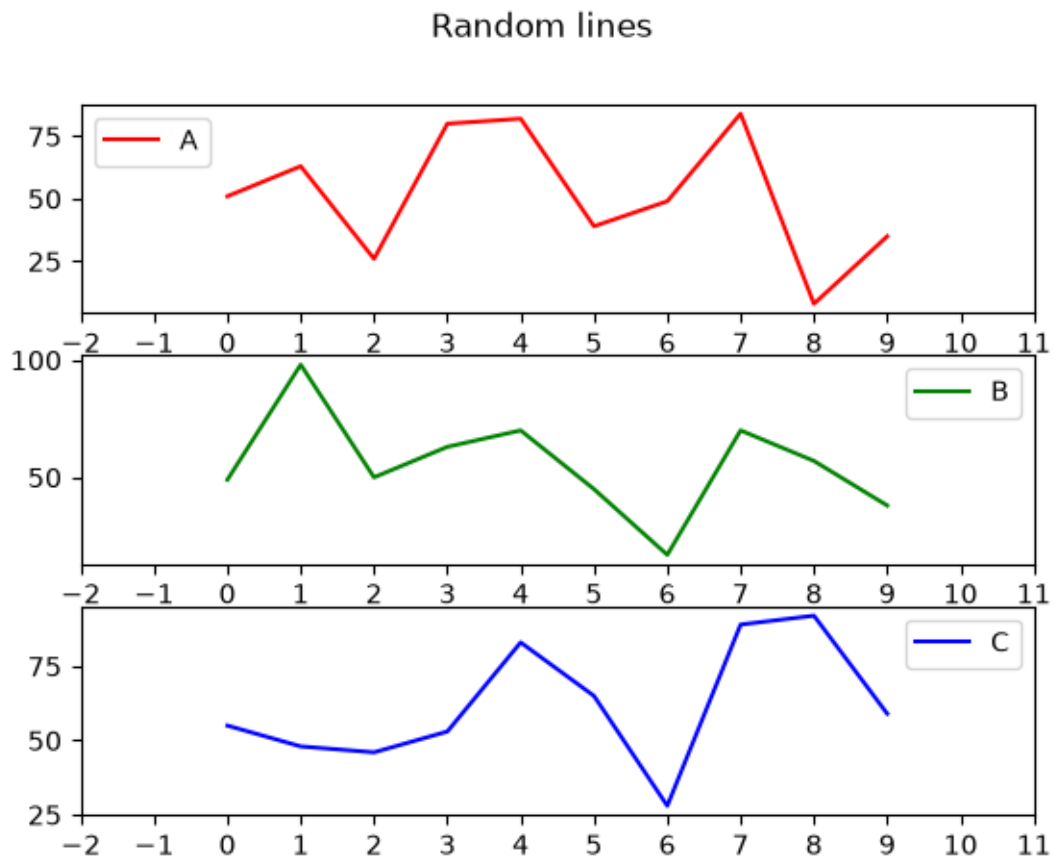
# Crear el DataFrame.
frame = pd.DataFrame(
    rand_matrix,
    columns=list("ABC")
)

# Crear tres subgráficos.
fig, axis = plt.subplots(3, 1)

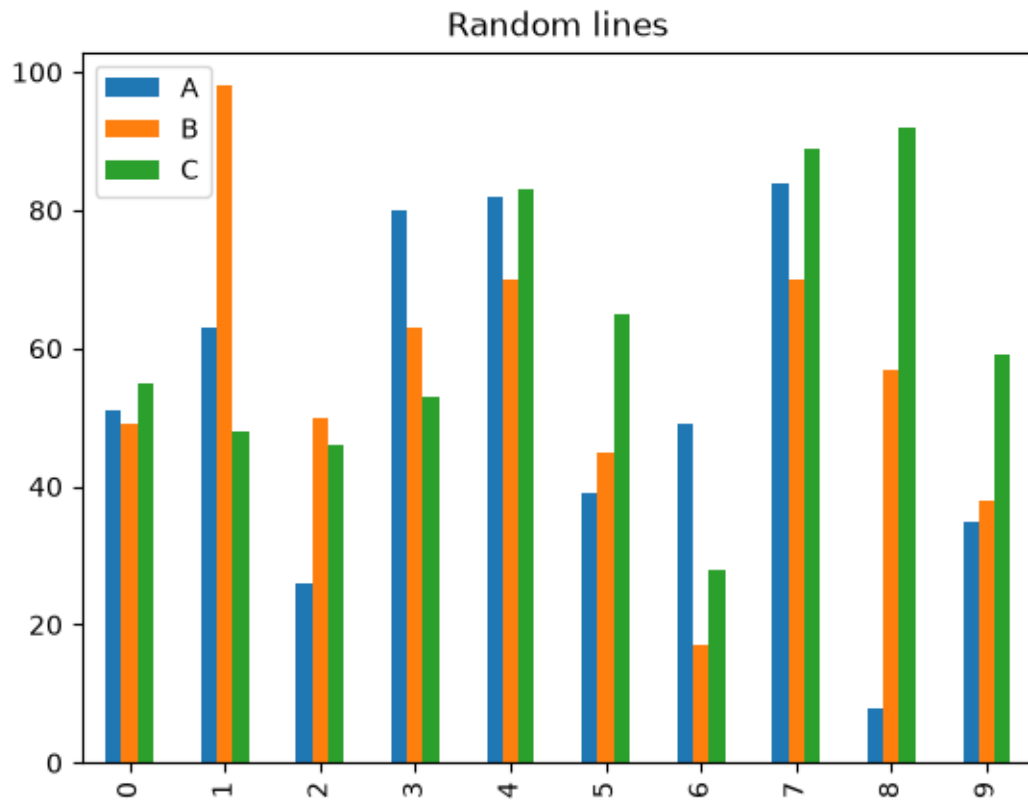
# Dibujar cada columna en un subplot diferente.
ax = frame.plot(
    style=["r-", "g-", "b-"],
    xticks=range(-2, 12),
    title="Random lines",
    subplots=True,
    ax=axis
)

# Mostrar la leyenda del primer gráfico.
ax[0].legend(loc="upper left")

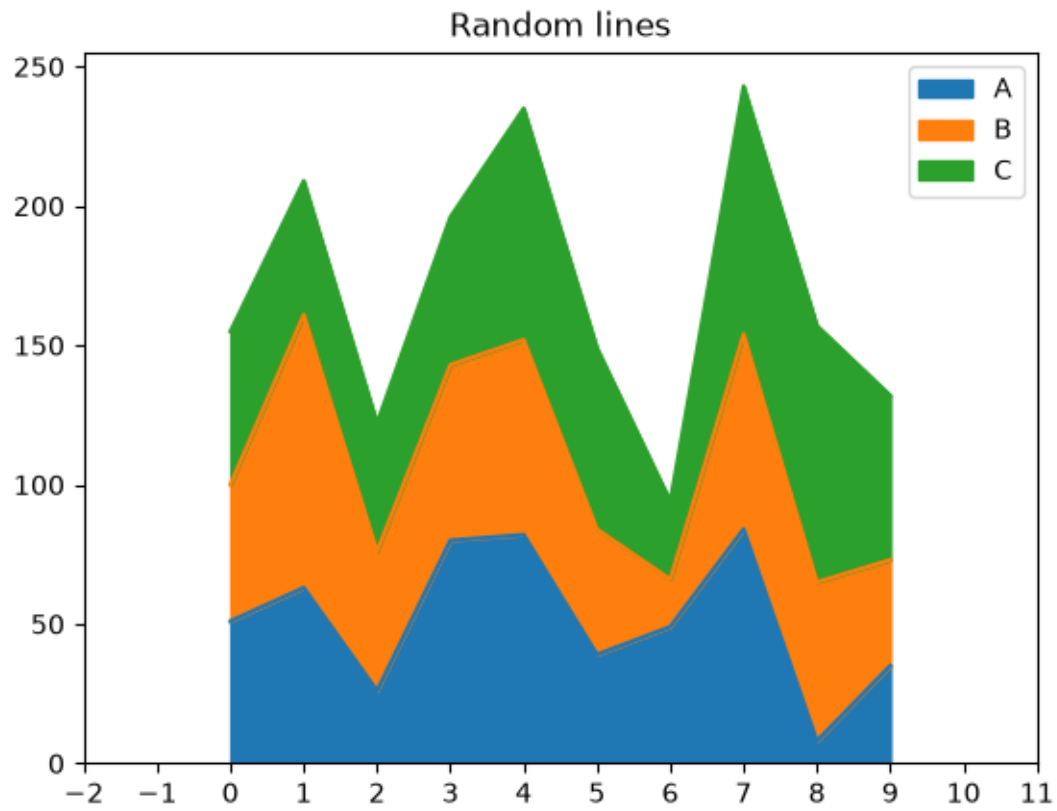
# Mostrar la figura.
plt.show()
```



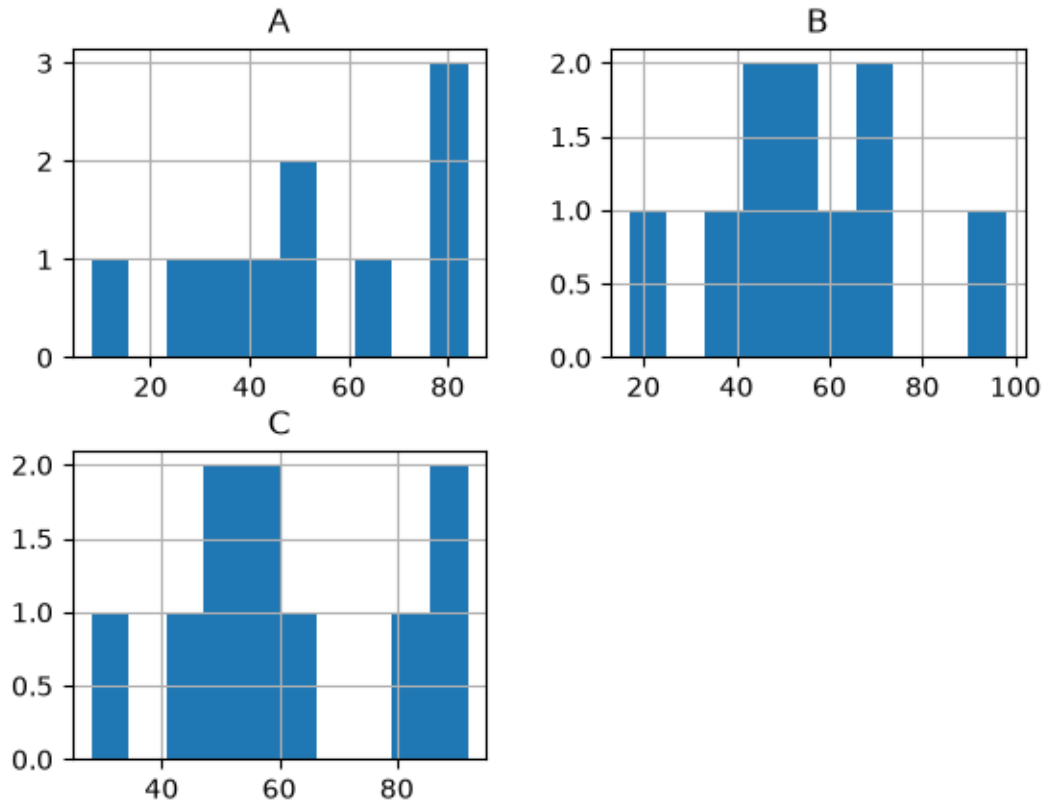
```
[26]: # =====  
# Gráfico de barras.  
# =====  
  
frame.plot(  
    kind="bar",  
    title="Random lines"  
)  
  
plt.show()
```



```
[27]: # =====  
# Gráfico de área.  
# =====  
  
frame.plot(  
    kind="area",  
    xticks=range(-2, 12),  
    title="Random lines"  
)  
  
plt.show()
```



```
[28]: # =====  
# Histograma.  
# =====  
  
frame.hist()  
  
plt.show()
```



```
[29]: # =====
# Gráfico de pastel (Pie Chart).
# =====

# Crear un DataFrame con información de algunos planetas.
df = pd.DataFrame(
    {
        "mass": [0.330, 4.87, 5.97],
        "radius": [2439.7, 6051.8, 6378.1]
    },
    index=["Mercury", "Venus", "Earth"]
)

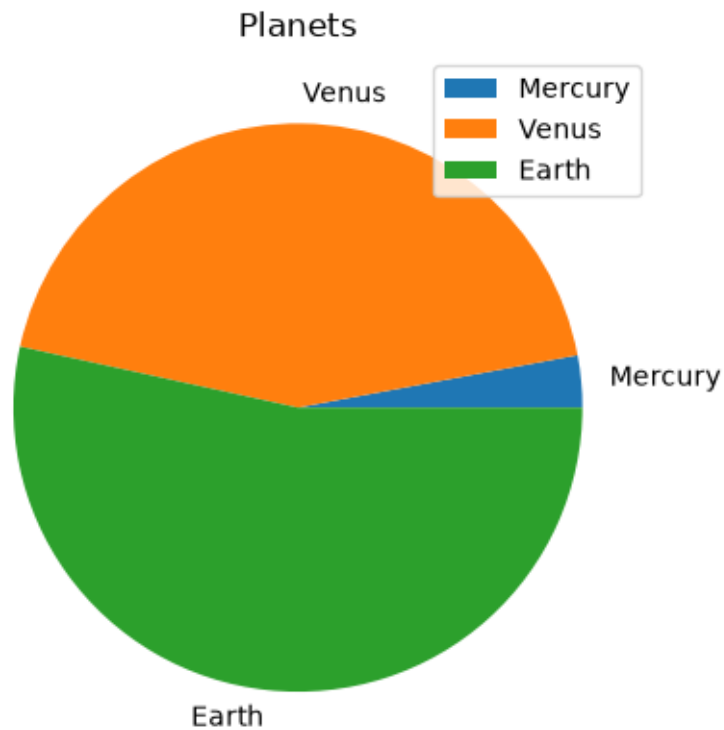
# Mostrar el DataFrame.
display(df)

# Crear el gráfico de pastel utilizando la columna "mass".
df.plot(
    y="mass",
    kind="pie",
    title="Planets"
```

```
)

# Mostrar el gráfico.
plt.show()
```

	mass	radius
Mercury	0.33	2439.7
Venus	4.87	6051.8
Earth	5.97	6378.1



Resumen

El método `plot()` de Pandas permite crear diferentes tipos de gráficos simplemente cambiando el valor del parámetro `kind`.

Algunos de los más utilizados son:

Tipo	Valor de kind
Línea	"line"
Barras	"bar"
Barras horizontales	"barh"
Área	"area"
Histograma	"hist"

Tipo	Valor de kind
Pastel	"pie"
Densidad	"density"
KDE	"kde"

7 7. Seaborn

Seaborn es una biblioteca de visualización de datos construida sobre **Matplotlib**, diseñada para crear gráficos estadísticos de forma sencilla y con una apariencia más atractiva.

7.1 Características

- Proporciona una abstracción sobre Matplotlib.
- Facilita la creación y personalización de gráficos.
- Es ideal para el análisis exploratorio de datos.
- Incluye varios conjuntos de datos de ejemplo listos para utilizar.

Los datasets disponibles pueden consultarse en:

<https://github.com/mwaskom/seaborn-data>

La documentación oficial está disponible en:

<https://seaborn.pydata.org/>

```
[32]: # =====
# Importar la biblioteca Seaborn.
# =====

import seaborn as sns
```

```
[33]: # =====
# Cargar un conjunto de datos de ejemplo incluido en Seaborn.
# =====

# Cargar el dataset "flights".
flights_data = sns.load_dataset("flights")

# Mostrar las primeras filas del DataFrame (opcional).
# display(flights_data)
```

```
[34]: # =====
# Crear un gráfico de dispersión tipo Swarm Plot.
# =====

# Crear el gráfico utilizando:
# - month: eje X
# - passengers: eje Y
sns.swarmplot(
```

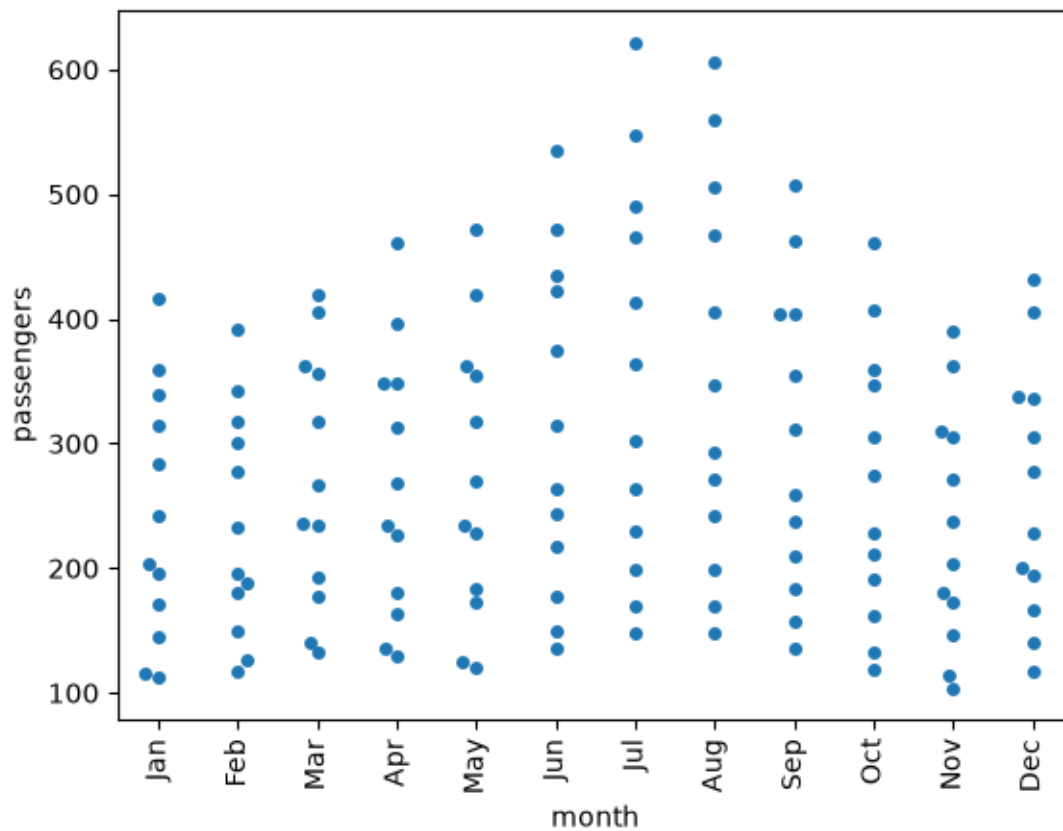
```

x="month",
y="passengers",
data=flights_data
)

# Rotar las etiquetas del eje X para mejorar su lectura.
plt.xticks(rotation=90)

# Mostrar el gráfico.
plt.show()

```



¿Qué es un Swarm Plot?

Un **Swarm Plot** representa cada observación del conjunto de datos como un punto individual, evitando que los puntos se superpongan.

Es especialmente útil cuando se desea visualizar:

- La distribución de los datos.
- La concentración de observaciones.
- Posibles valores atípicos (*outliers*).
- La relación entre una variable categórica y una variable numérica.

En este ejemplo:

- **month** representa los meses del año.
- **passengers** representa la cantidad de pasajeros registrada en cada mes.

7.1.1 Compatibilidad con Pandas

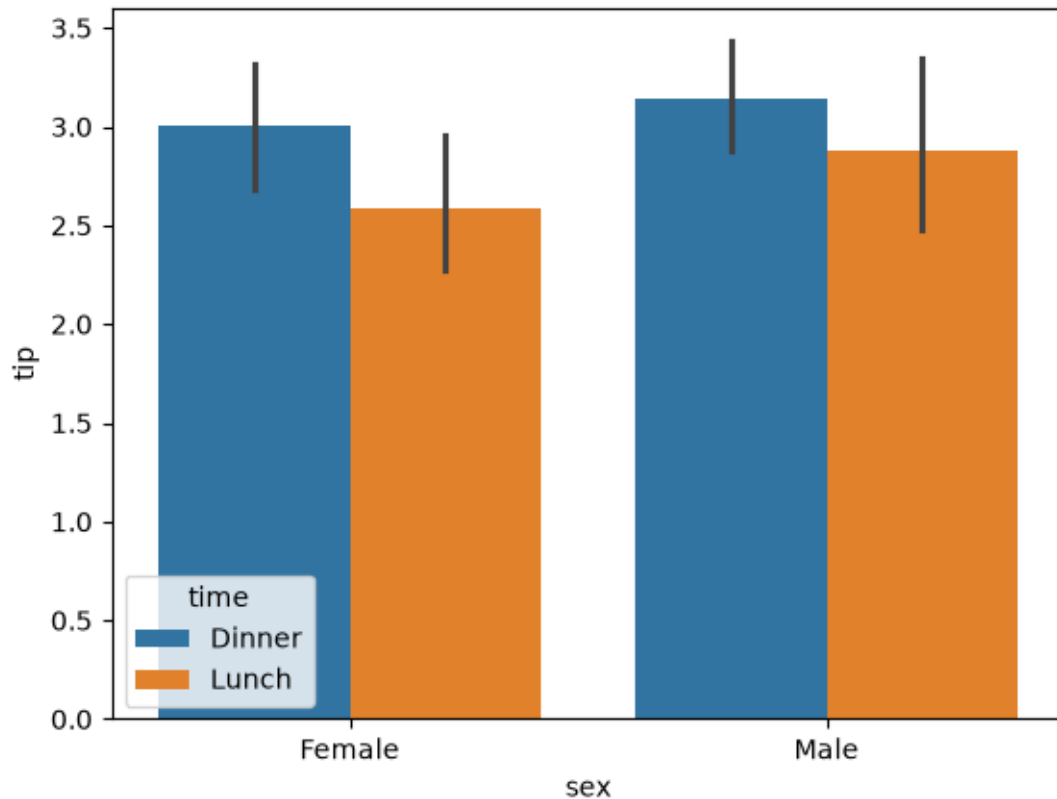
Una de las principales ventajas de Seaborn es su integración con **Pandas**. Esto permite utilizar directamente objetos **DataFrame** como fuente de datos para crear visualizaciones estadísticas.

En este ejemplo se carga el conjunto de datos **tips**, que contiene información sobre cuentas y propinas registradas en un restaurante.

```
[35]: # =====  
# Cargar un conjunto de datos desde un archivo CSV utilizando  
# Pandas.  
# =====  
  
# Leer el archivo CSV desde el repositorio oficial de Seaborn.  
tips = pd.read_csv(  
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"  
)  
  
# Mostrar una muestra aleatoria de cinco registros.  
tips.sample(5)
```

```
[35]:      total_bill   tip     sex smoker  day    time  size  
64         17.59  2.64    Male     No   Sat  Dinner     3  
73         25.28  5.00  Female     Yes   Sat  Dinner     2  
179        34.63  3.55    Male     Yes   Sun  Dinner     2  
103        22.42  3.48  Female     Yes   Sat  Dinner     2  
119        24.08  2.92  Female     No   Thur  Lunch     4
```

```
[36]: # =====  
# Crear un gráfico de barras utilizando Seaborn.  
# =====  
  
# Crear el gráfico de barras.  
sns.barplot(  
    x="sex",      # Variable categórica del eje X.  
    y="tip",      # Variable numérica del eje Y.  
    hue="time",   # Separar las barras según la hora del día.  
    data=tips     # DataFrame que contiene la información.  
)  
  
# Mostrar el gráfico.  
plt.show()
```



¿Qué hace `barplot()`?

El gráfico de barras de Seaborn calcula automáticamente un estadístico (por defecto, la **media**) para cada categoría y representa además un intervalo de confianza mediante una barra de error.

En este ejemplo:

- **sex** se utiliza como categoría del eje **X**.
- **tip** representa el valor promedio de las propinas en el eje **Y**.
- **hue="time"** divide cada categoría según el momento del servicio (*Lunch* o *Dinner*), utilizando un color diferente para cada grupo. > ¿Qué hace `barplot()`?

El gráfico de barras de Seaborn calcula automáticamente un estadístico (por defecto, la **media**) para cada categoría y representa además un intervalo de confianza mediante una barra de error.

En este ejemplo:

- **sex** se utiliza como categoría del eje **X**.
- **tip** representa el valor promedio de las propinas en el eje **Y**.
- **hue="time"** divide cada categoría según el momento del servicio (*Lunch* o *Dinner*), utilizando un color diferente para cada grupo.

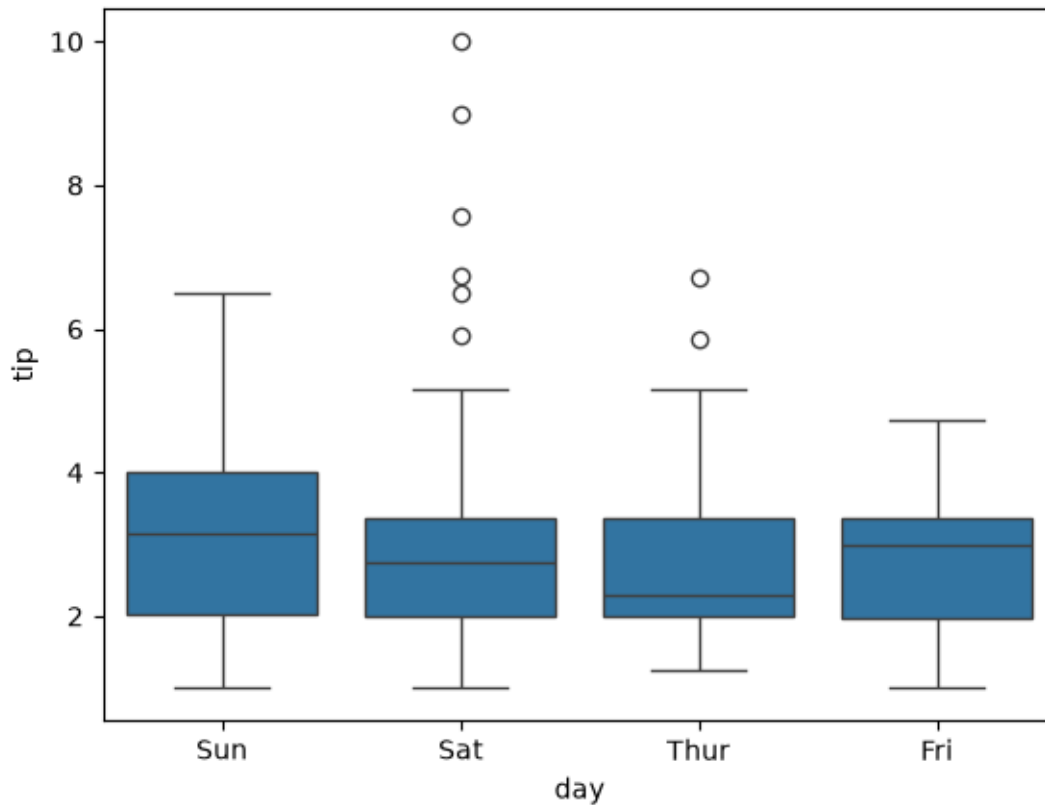
7.2 Boxplot (Diagrama de cajas)

El **Boxplot** o diagrama de cajas permite resumir la distribución de una variable numérica mediante sus cuartiles.

Este tipo de gráfico facilita la identificación de:

- Valor mínimo.
- Primer cuartil (Q1).
- Mediana (Q2).
- Tercer cuartil (Q3).
- Valor máximo.
- Posibles valores atípicos (*outliers*).

```
[37]: # =====  
# Crear un diagrama de cajas (Boxplot).  
# =====  
  
# Crear el gráfico utilizando:  
# - day: categorías del eje X.  
# - tip: valores numéricos del eje Y.  
sns.boxplot(  
    x="day",  
    y="tip",  
    data=tips  
)  
  
# También puede escribirse de la siguiente forma:  
# sns.boxplot(x=tips["day"], y=tips["tip"])  
  
# Mostrar el gráfico.  
plt.show()
```



7.3 7.1 Relaciones entre variables

El método `relplot()` permite analizar la relación entre una o varias variables de un conjunto de datos.

Dependiendo del parámetro `kind`, Seaborn puede generar diferentes tipos de gráficos, entre ellos:

- **Scatter Plot** (`kind="scatter"`)
- **Line Plot** (`kind="line"`)

Además, es posible representar varias variables utilizando colores, columnas o filas de manera automática.

```
[38]: # =====
# Cargar nuevamente el conjunto de datos "tips".
# =====

tips = sns.load_dataset("tips")

# Mostrar una muestra aleatoria del DataFrame.
tips.sample(5)
```

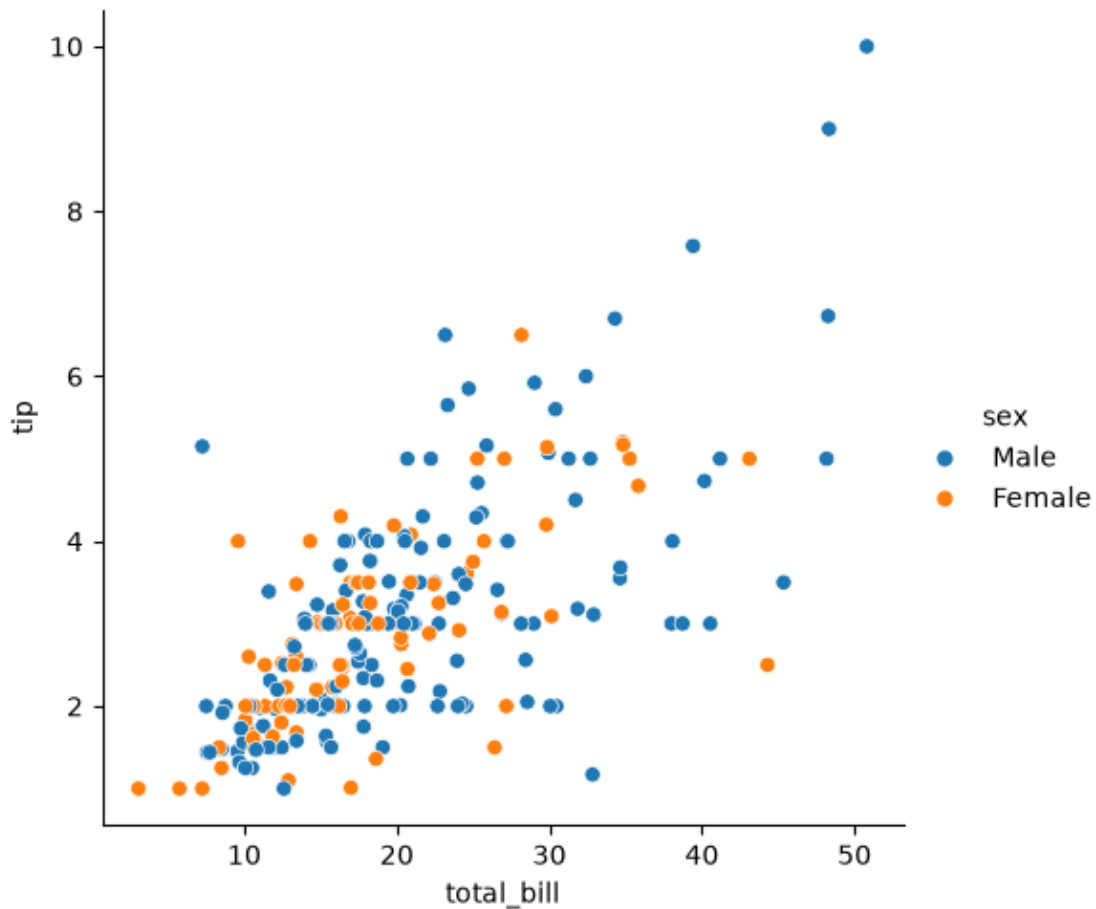
```
[38]:
```

	total_bill	tip	sex	smoker	day	time	size
59	48.27	6.73	Male	No	Sat	Dinner	4
3	23.68	3.31	Male	No	Sun	Dinner	2
184	40.55	3.00	Male	Yes	Sun	Dinner	2
160	21.50	3.50	Male	No	Sun	Dinner	4
109	14.31	4.00	Female	Yes	Sat	Dinner	2

```
[39]: # =====
# Relación entre el total de la cuenta y la propina.
# =====

sns.relplot(
    x="total_bill",
    y="tip",
    hue="sex",
    data=tips,
    kind="scatter"
)

# Mostrar el gráfico.
plt.show()
```

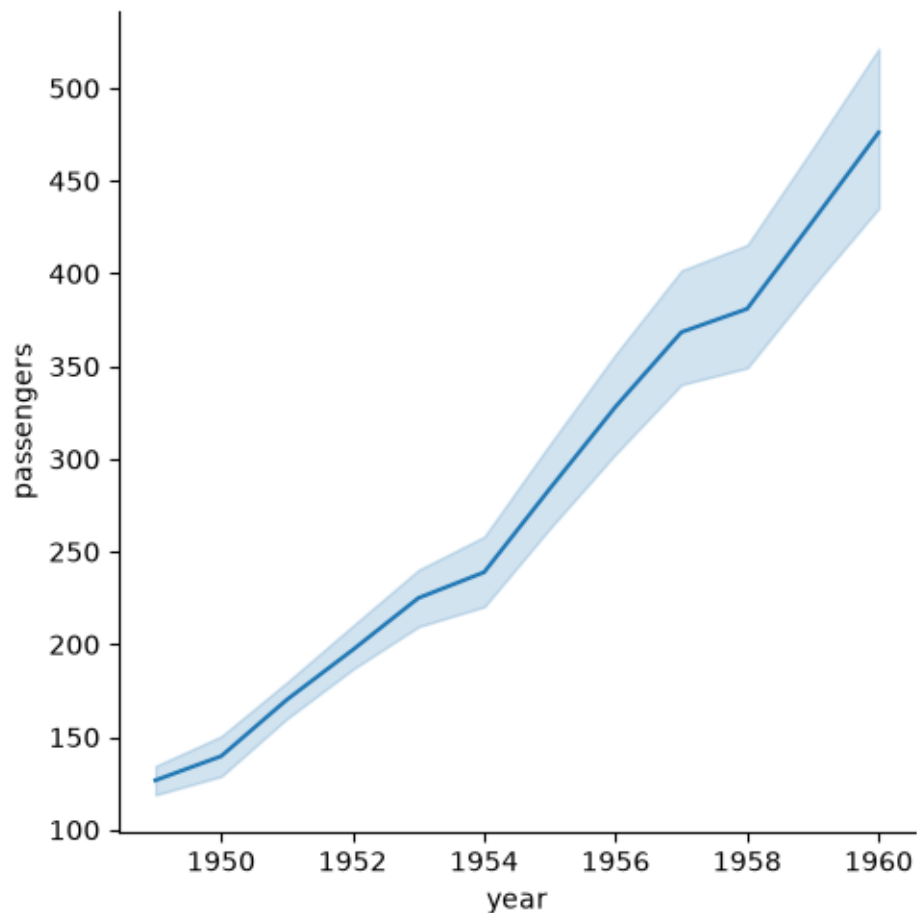


```
[40]: # =====
# Representar la evolución del número de pasajeros
# a lo largo de los años.
# =====

# Cargar el conjunto de datos "flights".
data = sns.load_dataset("flights")

# Crear un gráfico de líneas.
sns.relplot(
    x="year",
    y="passengers",
    kind="line",
    data=data
)

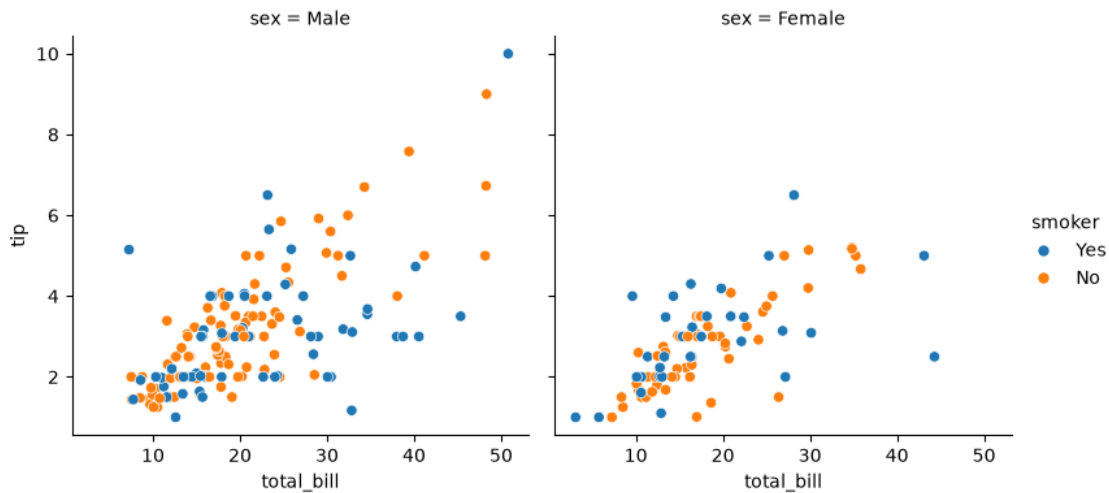
# Mostrar el gráfico.
plt.show()
```



```
[41]: # =====
# Relacionar varias variables automáticamente.
# =====

sns.relplot(
    x="total_bill",
    y="tip",
    hue="smoker",      # Color según si el cliente fuma.
    col="sex",         # Un gráfico por sexo.
    height=4,         # Altura de cada gráfico.
    kind="scatter",    # Tipo de gráfico.
    data=tips
)

# Mostrar el gráfico.
plt.show()
```



¿Qué permite hacer `relplot()`?

`relplot()` es una de las funciones más potentes de Seaborn, ya que permite representar simultáneamente varias variables en un mismo conjunto de gráficos.

Algunos parámetros importantes son:

Parámetro	Función
x	Variable del eje X.
y	Variable del eje Y.
hue	Diferencia las categorías mediante colores.

Parámetro	Función
<code>col</code>	Crea una columna de gráficos para cada categoría.
<code>row</code>	Crea una fila de gráficos para cada categoría.
<code>kind</code>	Tipo de gráfico (<code>scatter</code> , <code>line</code> , etc.).
<code>height</code>	Altura de cada subgráfico.

7.4 7.2 Distribución de un conjunto de datos

Seaborn proporciona diversas funciones para analizar la distribución de una o varias variables.

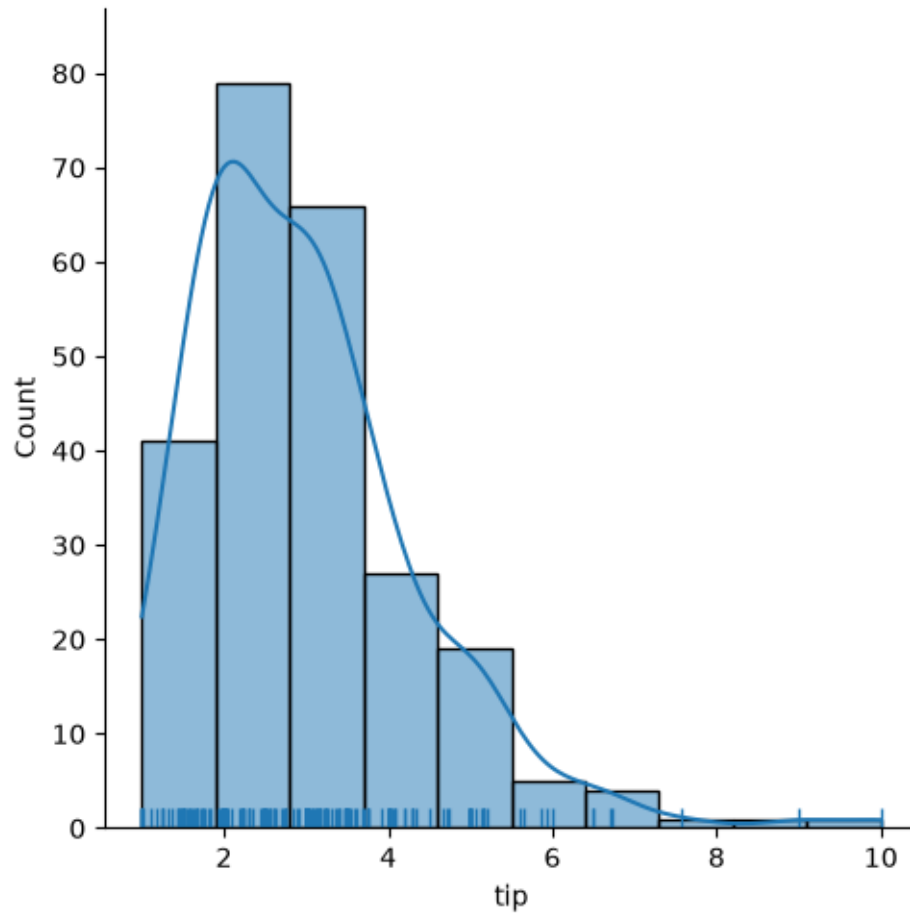
Estas visualizaciones permiten comprender la forma en que se distribuyen los datos, identificar patrones, detectar valores atípicos y estudiar la relación entre diferentes variables.

```
[42]: # =====
# Histograma con curva de densidad (Density Plot).
# =====

# kde -> Muestra la curva de densidad (Kernel Density Estimate).
# bins -> Número de intervalos del histograma.
# rug -> Dibuja pequeñas marcas indicando cada observación.

sns.displot(
    tips["tip"],
    kde=True,
    bins=10,
    rug=True
)

# Mostrar el gráfico.
plt.show()
```

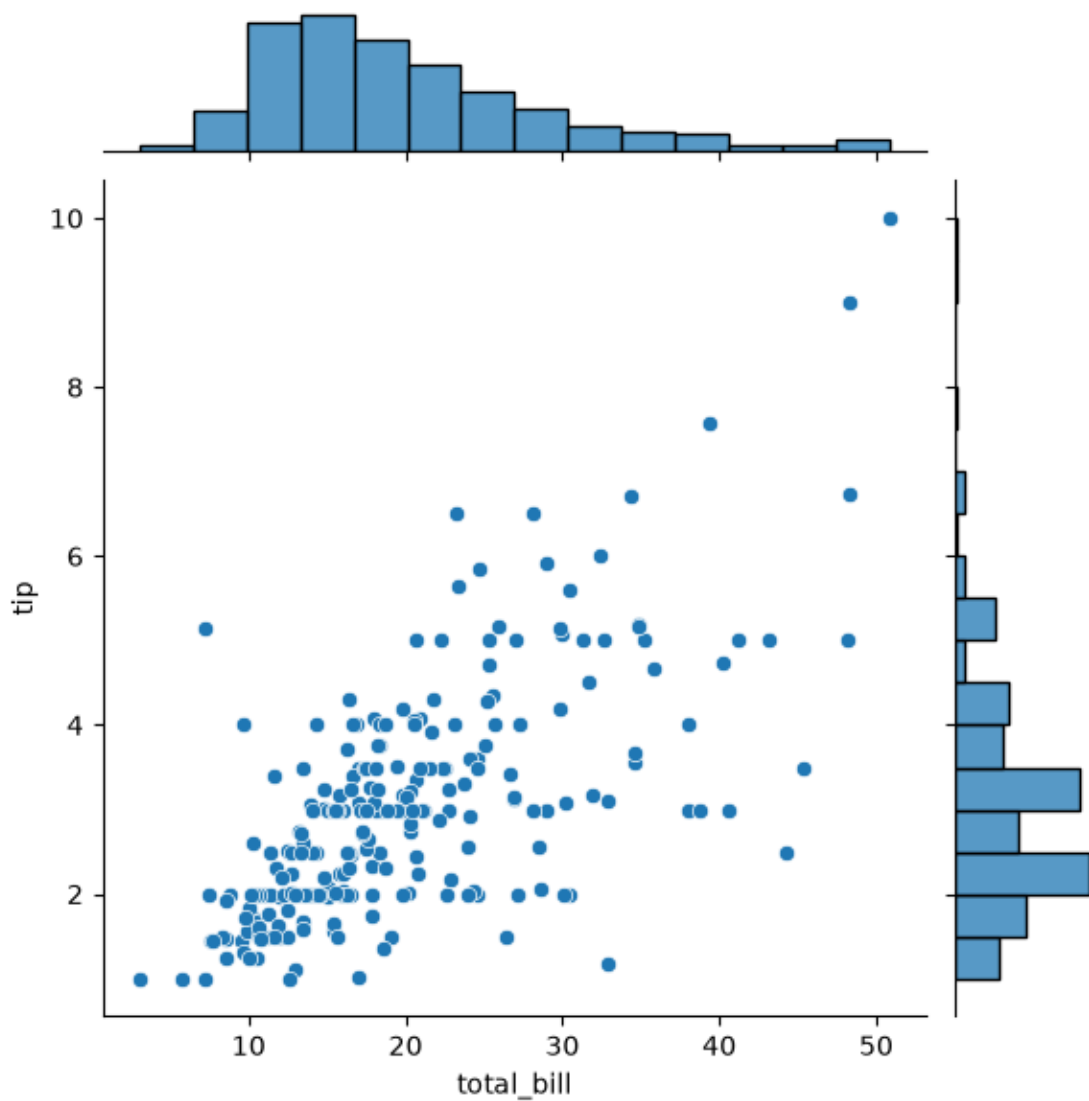
7.4.1 Distribución bivariada

Cuando se desea estudiar la relación entre dos variables numéricas, Seaborn ofrece el método `jointplot()`, que combina un gráfico principal con histogramas marginales para cada variable.

```
[43]: # =====
# Distribución conjunta de dos variables.
# =====

sns.jointplot(
    x="total_bill",
    y="tip",
    data=tips
)

# Mostrar el gráfico.
plt.show()
```



```
[44]: # =====
# Distribución conjunta mediante curvas de densidad.
# =====

# Cargar el conjunto de datos Iris.
iris = sns.load_dataset("iris")

# Mostrar las primeras filas del DataFrame.
display(iris.head())

# Mostrar las especies existentes.
display(iris["species"].unique())
```

```

# Crear el gráfico de densidad bivariada.
sns.jointplot(
    x="petal_length",
    y="petal_width",
    kind="kde",
    data=iris
)

# Mostrar el gráfico.
plt.show()

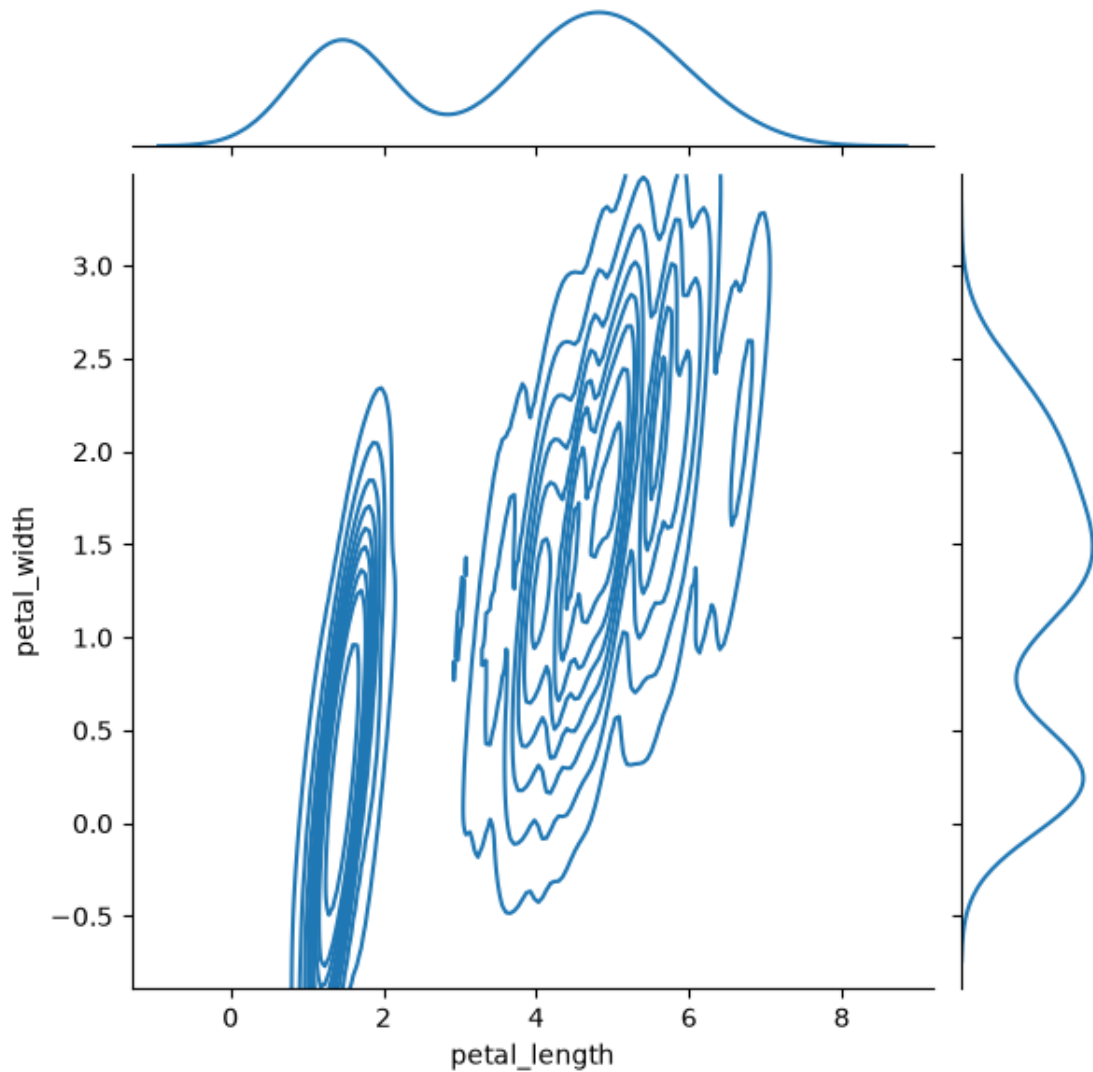
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```

<StringArray>
['setosa', 'versicolor', 'virginica']
Length: 3, dtype: str

```



7.4.2 Pair Plot

El método `pairplot()` genera automáticamente una matriz de gráficos que muestra la relación entre todas las variables numéricas de un conjunto de datos.

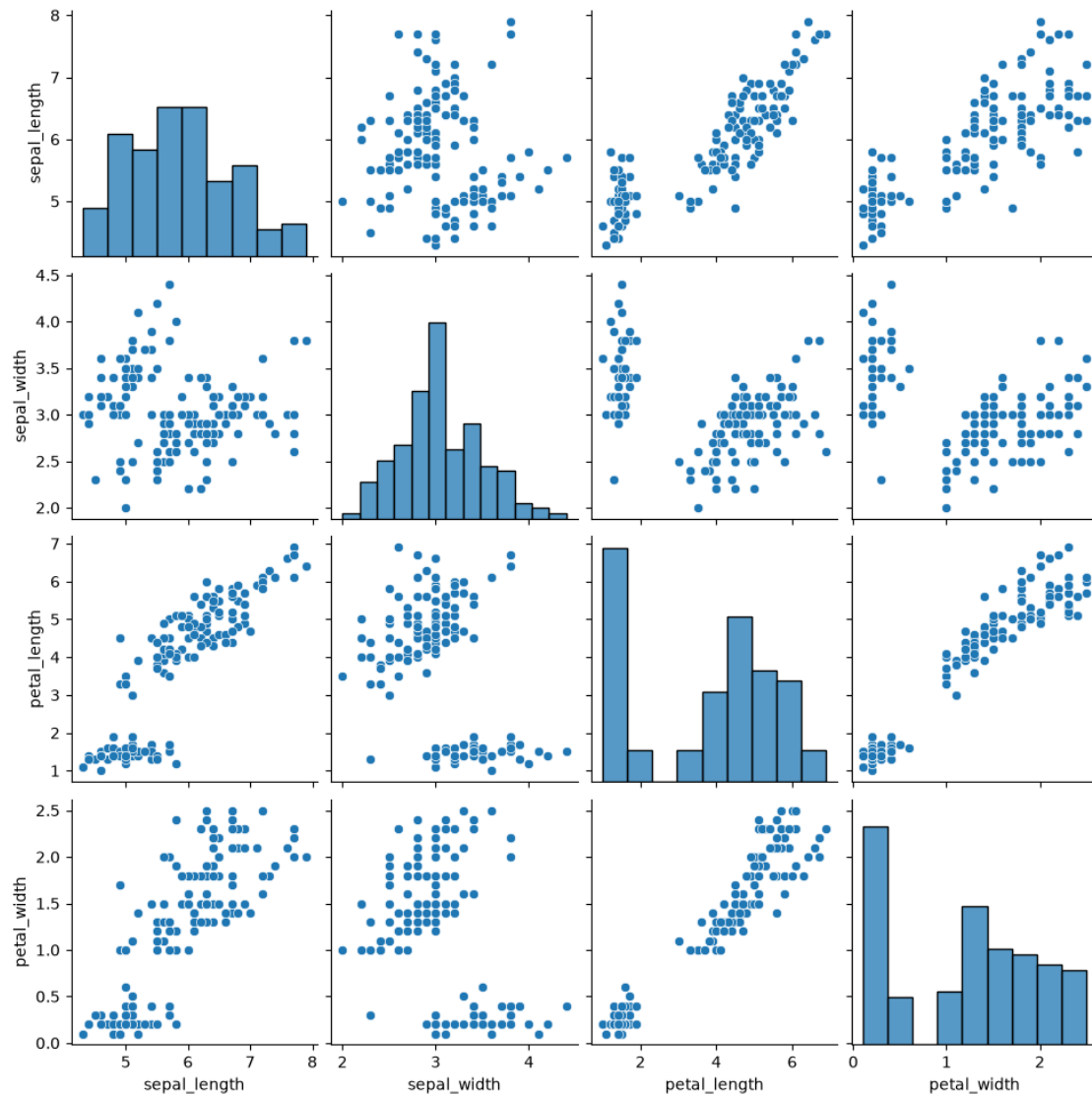
Es una herramienta muy utilizada durante el análisis exploratorio de datos (EDA).

```
[45]: # =====
# Relación entre todas las variables numéricas.
# =====

sns.pairplot(iris)

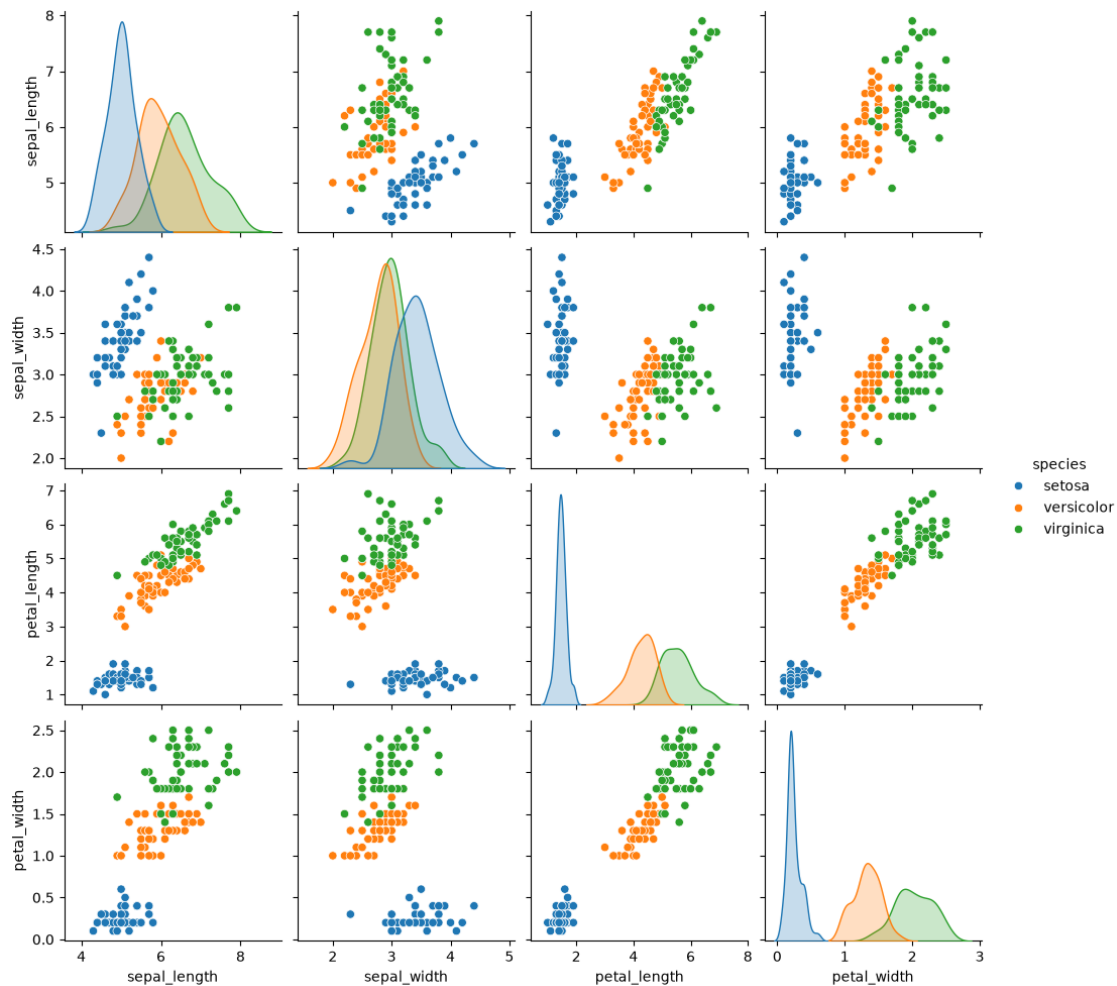
# Mostrar el gráfico.
```

```
plt.show()
```



```
[46]: # =====  
# Pair Plot diferenciando las especies mediante colores.  
# =====  
  
sns.pairplot(  
    iris,  
    hue="species"  
)  
  
# Mostrar el gráfico.
```

```
plt.show()
```



7.4.3 Gráfico de dispersión por clases utilizando Matplotlib

En este ejemplo se utiliza Matplotlib para representar las especies del conjunto de datos **Iris**, asignando un color diferente a cada clase.

Este procedimiento permite comprender cómo preparar manualmente los datos antes de construir una visualización.

```
[47]: # =====  
# Gráfico de dispersión clasificando los datos por especie.  
# =====  
  
# Cargar nuevamente el conjunto de datos Iris.  
iris = sns.load_dataset("iris")
```

```

# -----
# Preparar la información para cada especie.
# -----

data = []

colors = ["red", "green", "blue"]

groups = ["setosa", "versicolor", "virginica"]

columns = ["petal_length", "petal_width"]

# Obtener los datos correspondientes a cada especie.
for group in groups:

    data.append(
        iris[
            iris["species"] == group
        ][columns].values
    )

# -----
# Crear la figura.
# -----

fig = plt.figure()

ax = fig.add_subplot()

# Dibujar cada especie con un color diferente.
for index, values in enumerate(data):

    # Separar las columnas en los ejes X e Y.
    x = values[:, 0]
    y = values[:, 1]

    ax.scatter(
        x,
        y,
        alpha=0.4,
        c=colors[index],
        edgecolors="none",
        s=30,
        label=groups[index]
    )

# Etiquetar los ejes.

```

```

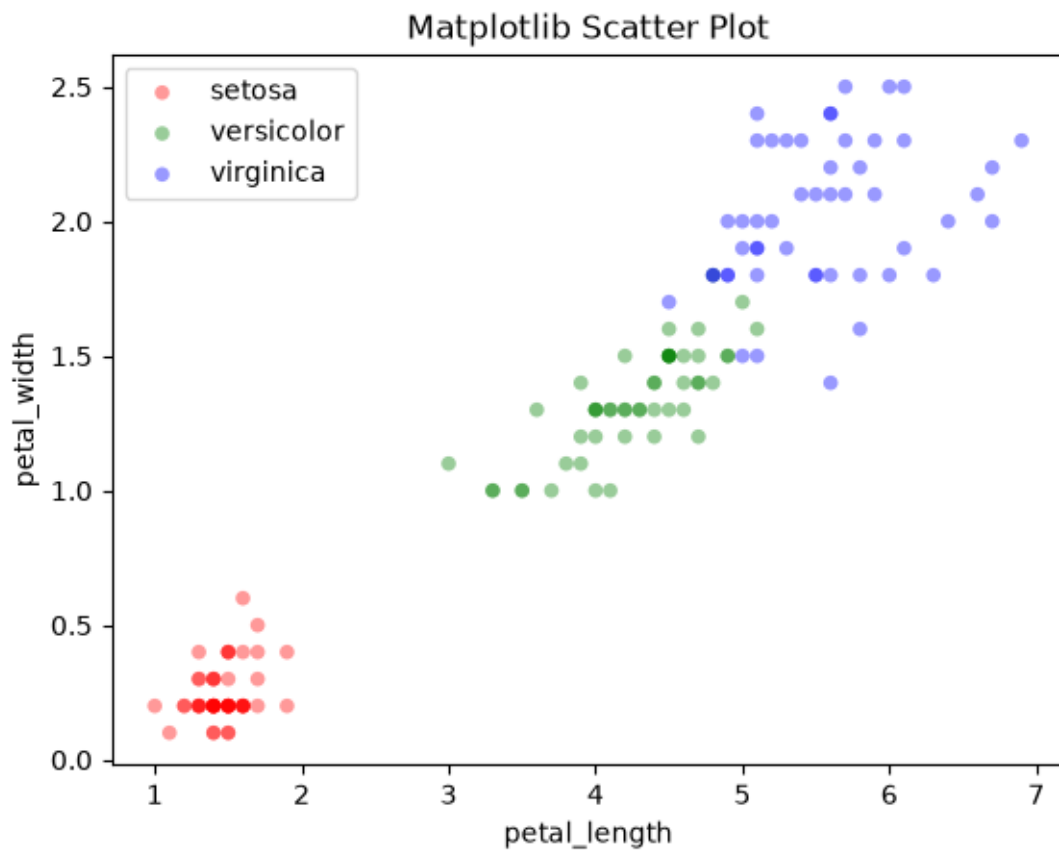
ax.set_xlabel(columns[0])
ax.set_ylabel(columns[1])

# Mostrar la leyenda.
plt.legend(loc="best")

# Agregar un título.
plt.title("Matplotlib Scatter Plot")

# Mostrar el gráfico.
plt.show()

```



Resumen de las funciones utilizadas

Función	Propósito
<code>displot()</code>	Mostrar la distribución de una variable mediante histogramas y curvas de densidad.
<code>jointplot()</code>	Analizar la relación entre dos variables.
<code>pairplot()</code>	Visualizar automáticamente la relación entre todas las variables de un conjunto de datos.

Función	Propósito
<code>scatter()</code>	Crear gráficos de dispersión personalizados con Matplotlib.

7.5 7.3 Visualización de correlaciones

La correlación permite analizar la relación existente entre dos variables numéricas.

Seaborn proporciona funciones que permiten representar gráficamente dicha relación mediante una **recta de regresión**, facilitando la identificación de tendencias, asociaciones y posibles patrones en los datos.

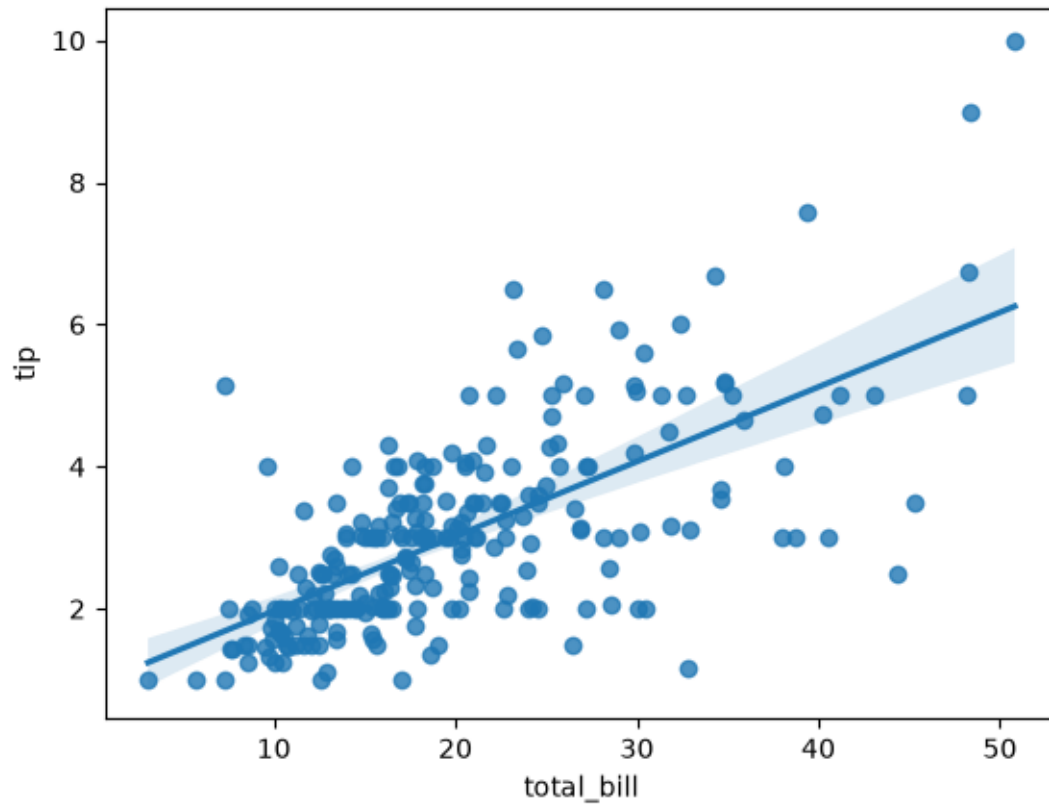
7.5.1 Aplicaciones

- Analizar la relación entre variables.
- Estimar tendencias mediante regresión lineal.
- Visualizar intervalos de confianza.
- Comparar relaciones entre diferentes grupos de datos.

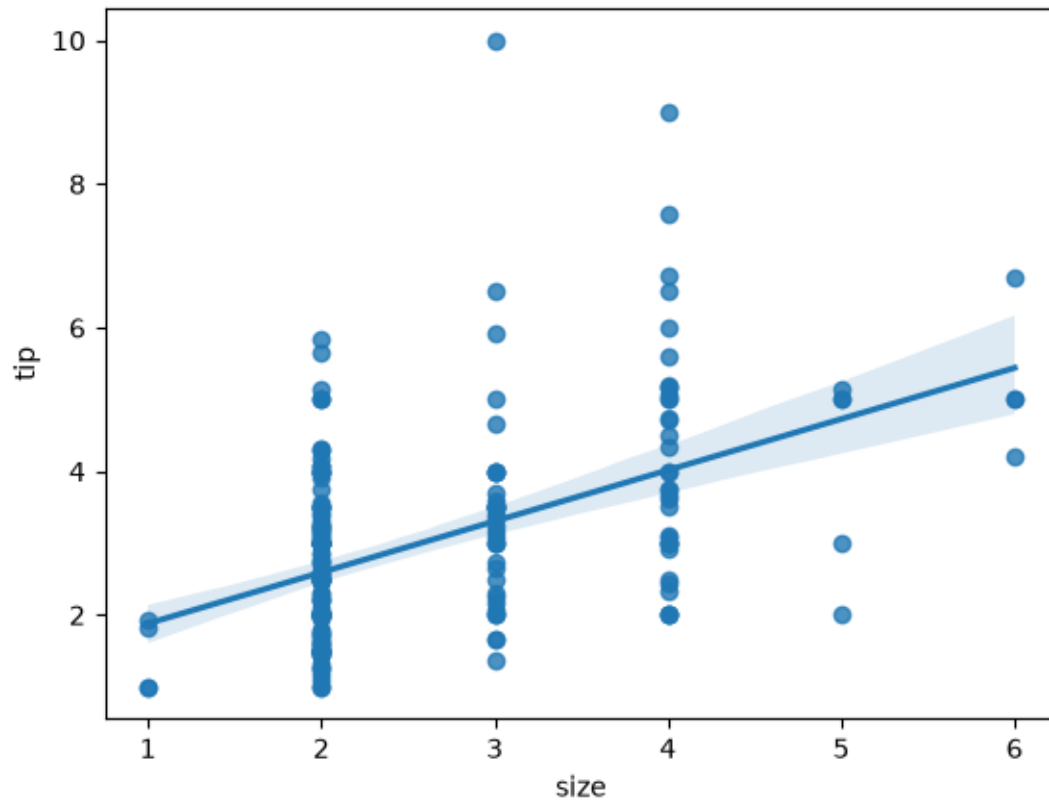
```
[48]: # =====
# Correlación entre el total de la cuenta y la propina.
# Se muestra la recta de regresión y el intervalo de confianza.
# =====

sns.regplot(
    x="total_bill",
    y="tip",
    data=tips
)

# Mostrar el gráfico.
plt.show()
```



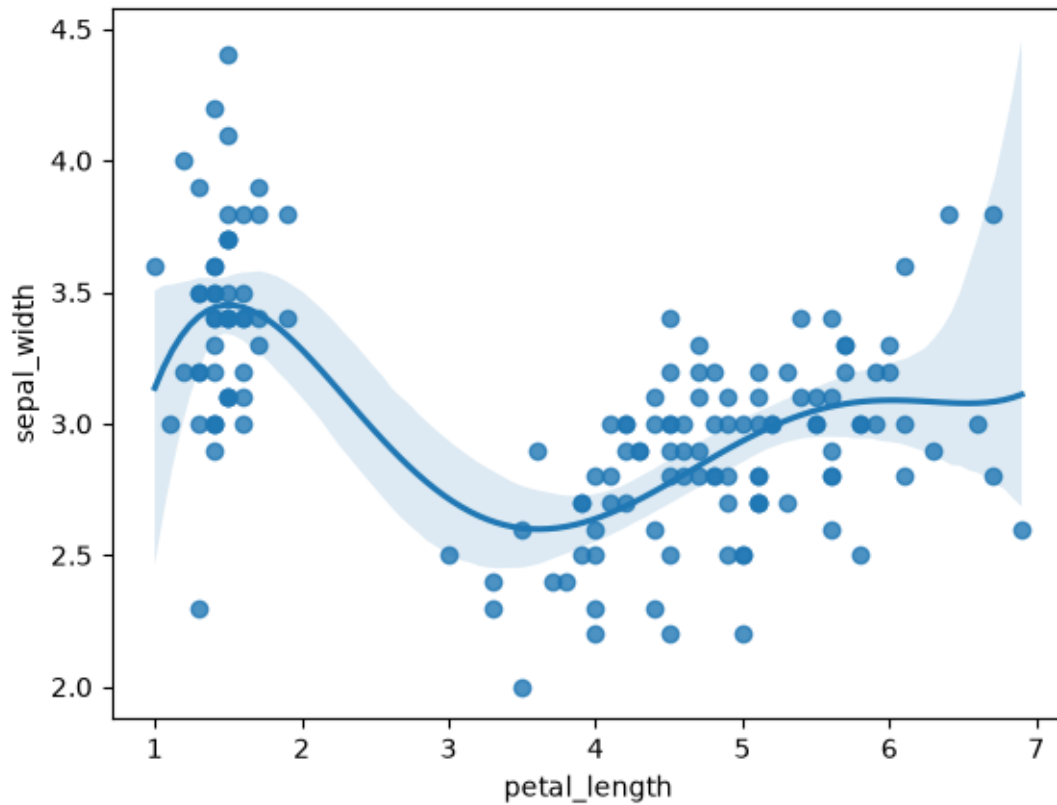
```
[49]: # =====  
# Regresión utilizando una variable discreta.  
# En este caso "size" representa el número de personas.  
# =====  
  
sns.regplot(  
    x="size",  
    y="tip",  
    data=tips  
)  
  
# Mostrar el gráfico.  
plt.show()
```



```
[50]: # =====
# Regresión polinómica.
# El parámetro "order" indica el grado del polinomio.
# =====

sns.regplot(
    x="petal_length",
    y="sepal_width",
    data=iris,
    order=5
)

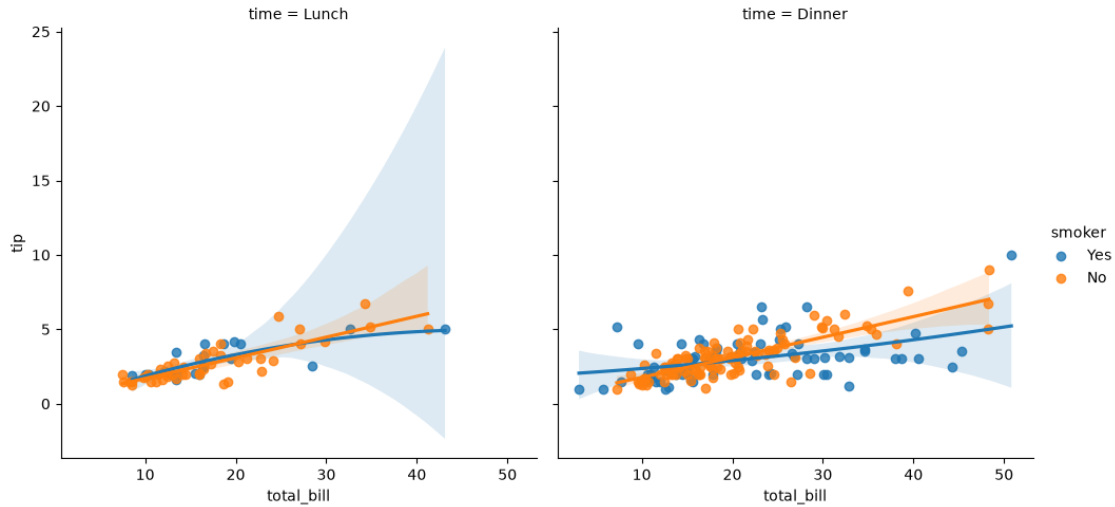
# Mostrar el gráfico.
plt.show()
```



```
[51]: # =====
# Comparar correlaciones entre varios grupos de datos.
# =====

sns.lmplot(
    x="total_bill",
    y="tip",
    hue="smoker",    # Diferenciar fumadores y no fumadores.
    col="time",      # Crear un gráfico por cada momento del día.
    data=tips,
    order=2          # Ajustar una regresión polinómica de grado 2.
)

# Mostrar el gráfico.
plt.show()
```



Funciones utilizadas

Función	Descripción
<code>regplot()</code>	Representa la relación entre dos variables mediante una recta de regresión e intervalo de confianza.
<code>lmplot()</code>	Permite realizar regresiones lineales separando automáticamente los datos por categorías mediante parámetros como <code>hue</code> , <code>row</code> o <code>col</code> .
<code>order</code>	Define el grado del polinomio utilizado para ajustar la regresión.

8. Ejercicios

En esta sección se aplican los conocimientos adquiridos sobre **Matplotlib** y **Seaborn** mediante ejercicios prácticos de visualización de datos.

8.1 Matplotlib

8.1.1 Objetivos

- Crear una lista de años desde **2000** hasta **2020**.
- Generar cuatro secuencias de datos aleatorios.
- Representar las cuatro series en un mismo gráfico.
- Representar las mismas series en subgráficos independientes.
- Agregar títulos, etiquetas, leyendas y anotaciones.

```
[52]: # =====
# Generar los datos para los ejercicios de Matplotlib.
# =====

# Crear la lista de años desde el 2000 hasta el 2020.
years = np.arange(2000, 2021)

# Generar cuatro secuencias de datos aleatorios.
y0 = np.random.randint(50, 250, size=len(years))
y1 = np.random.randint(50, 250, size=len(years))
y2 = np.random.randint(50, 250, size=len(years))
y3 = np.random.randint(50, 250, size=len(years))
```

```
[53]: # =====
# Representar las cuatro series en un mismo gráfico.
# =====

fig, ax = plt.subplots(figsize=(10, 5))

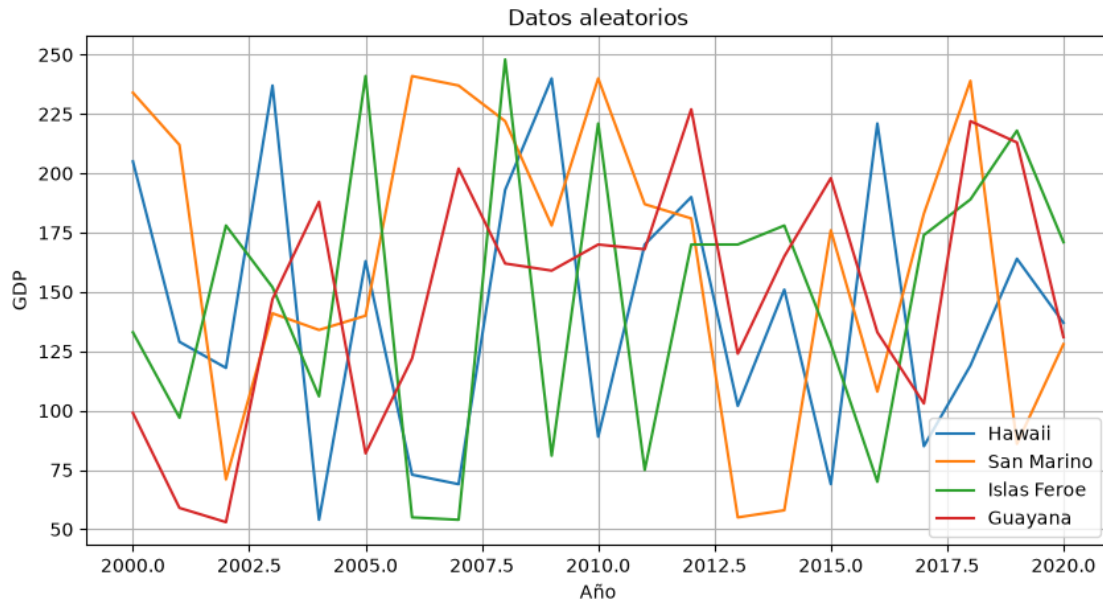
ax.plot(years, y0, label="Hawaii")
ax.plot(years, y1, label="San Marino")
ax.plot(years, y2, label="Islas Feroe")
ax.plot(years, y3, label="Guayana")

# Configurar el gráfico.
ax.set_title("Datos aleatorios")
ax.set_xlabel("Año")
ax.set_ylabel("GDP")

# Mostrar la leyenda.
ax.legend()

# Agregar una cuadrícula.
plt.grid()

# Mostrar el gráfico.
plt.show()
```



```
[54]: # =====
# Representar las cuatro series en subgráficos.
# =====

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

series = [y0, y1, y2, y3]

labels = [
    "Hawaii",
    "San Marino",
    "Islas Feroe",
    "Guayana"
]

for ax, values, label in zip(axes.flat, series, labels):

    ax.plot(years, values)

    ax.set_title("Datos aleatorios")

    ax.set_xlabel("Año")

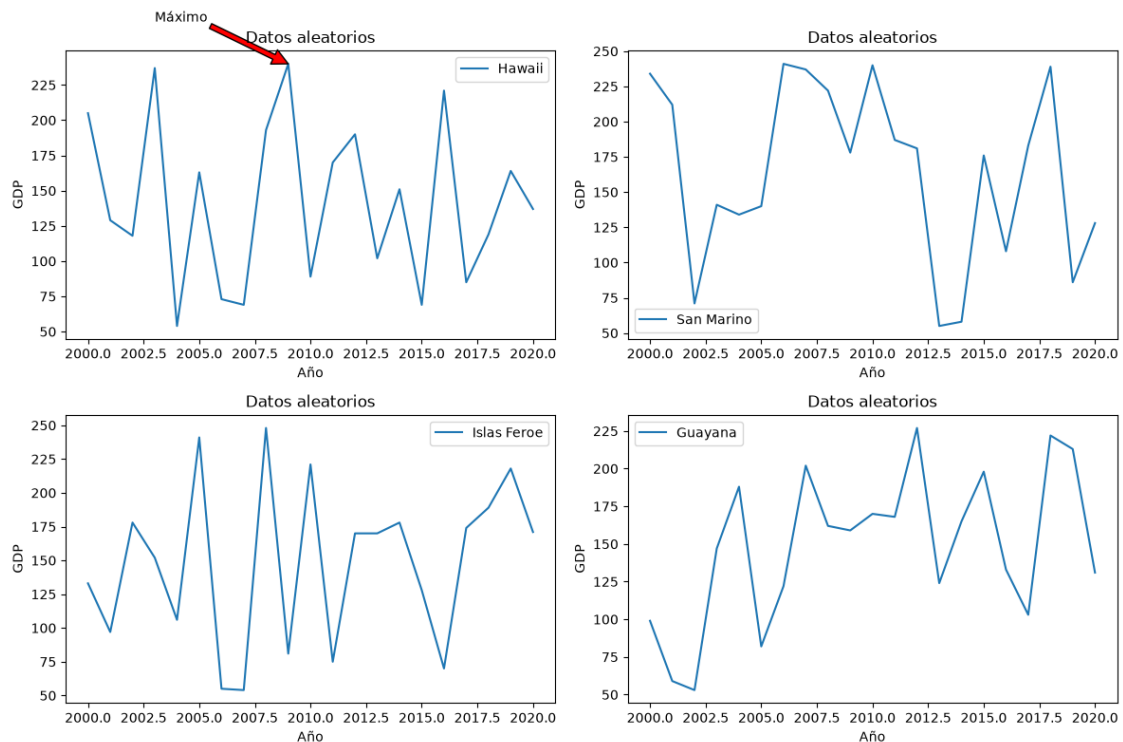
    ax.set_ylabel("GDP")

    ax.legend([label])
```

```
# Agregar una anotación al primer gráfico.
axes[0, 0].annotate(
    "Máximo",
    xy=(years[np.argmax(y0)], np.max(y0)),
    xytext=(2003, np.max(y0) + 30),
    arrowprops=dict(facecolor="red")
)

plt.tight_layout()

plt.show()
```



8.2 8.2 Seaborn

Para estos ejercicios se utilizará un conjunto de datos diferente a:

- tips
- iris
- flights

En este caso se empleará el dataset **penguins**, incluido en Seaborn.

```
[55]: # =====
# Cargar el conjunto de datos.
```



```
# =====

penguins = sns.load_dataset("penguins")

# Mostrar las primeras filas.
display(penguins.head())
```

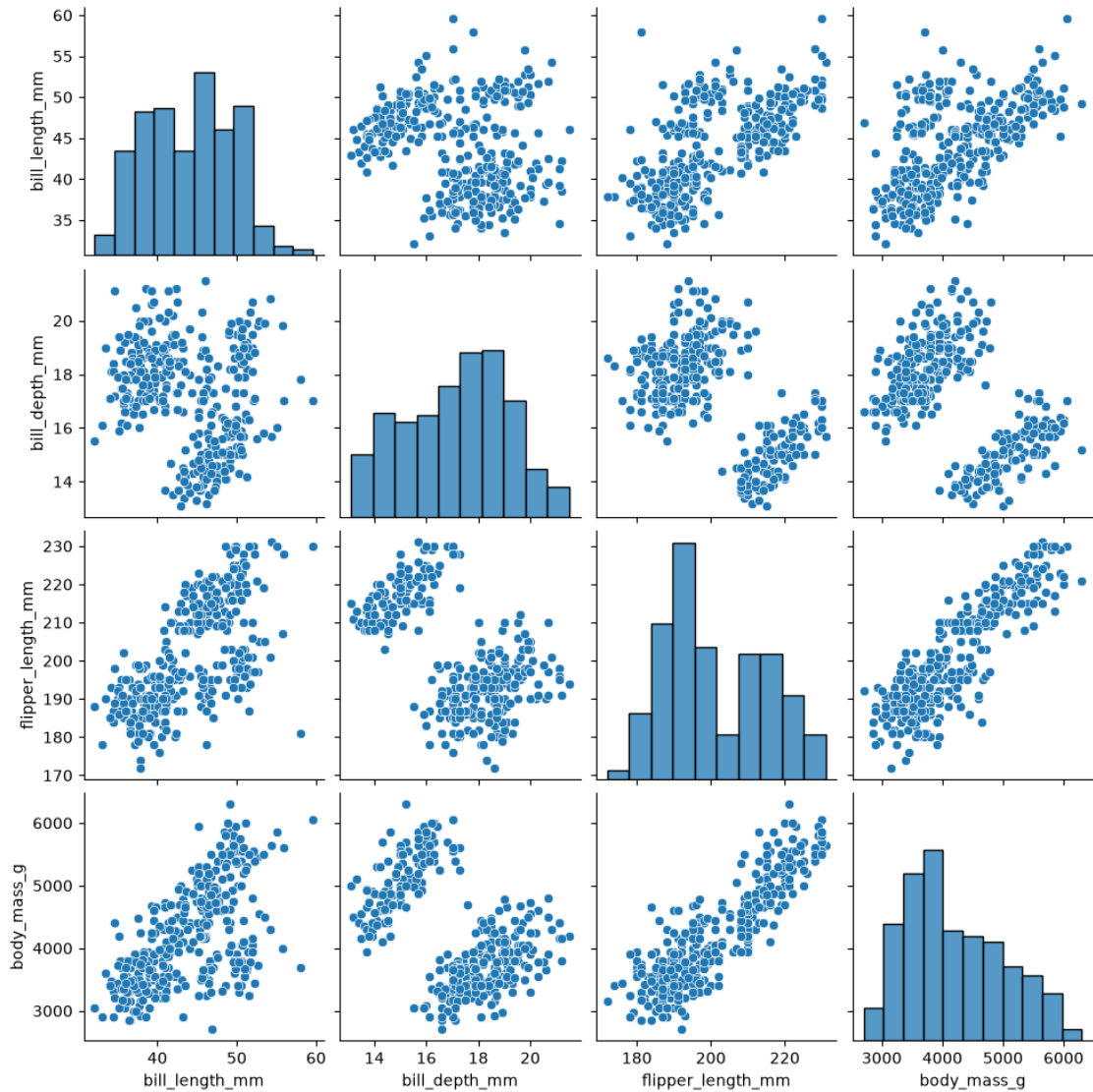
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	NaN	NaN
4	3450.0	Female

```
[56]: # =====
# Relación entre todas las variables.
# =====

sns.pairplot(penguins)

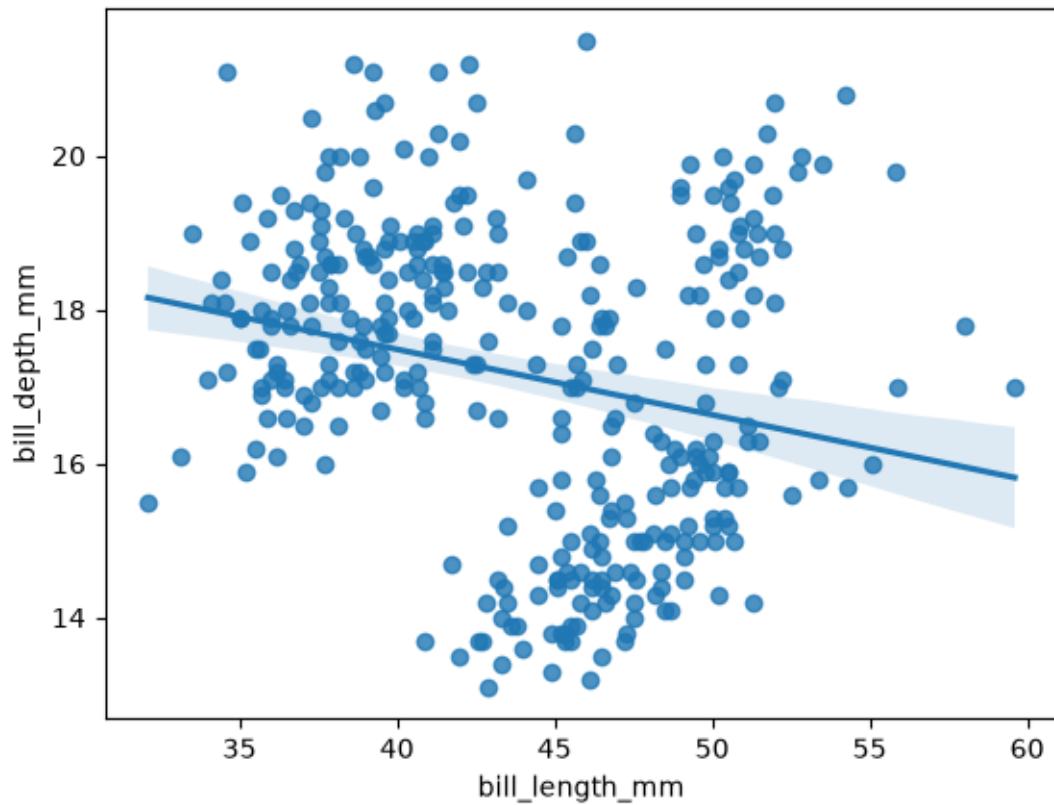
plt.show()
```



```
[57]: # =====
# Demostrar la correlación entre dos variables.
# =====

sns.regplot(
    x="bill_length_mm",
    y="bill_depth_mm",
    data=penguins
)

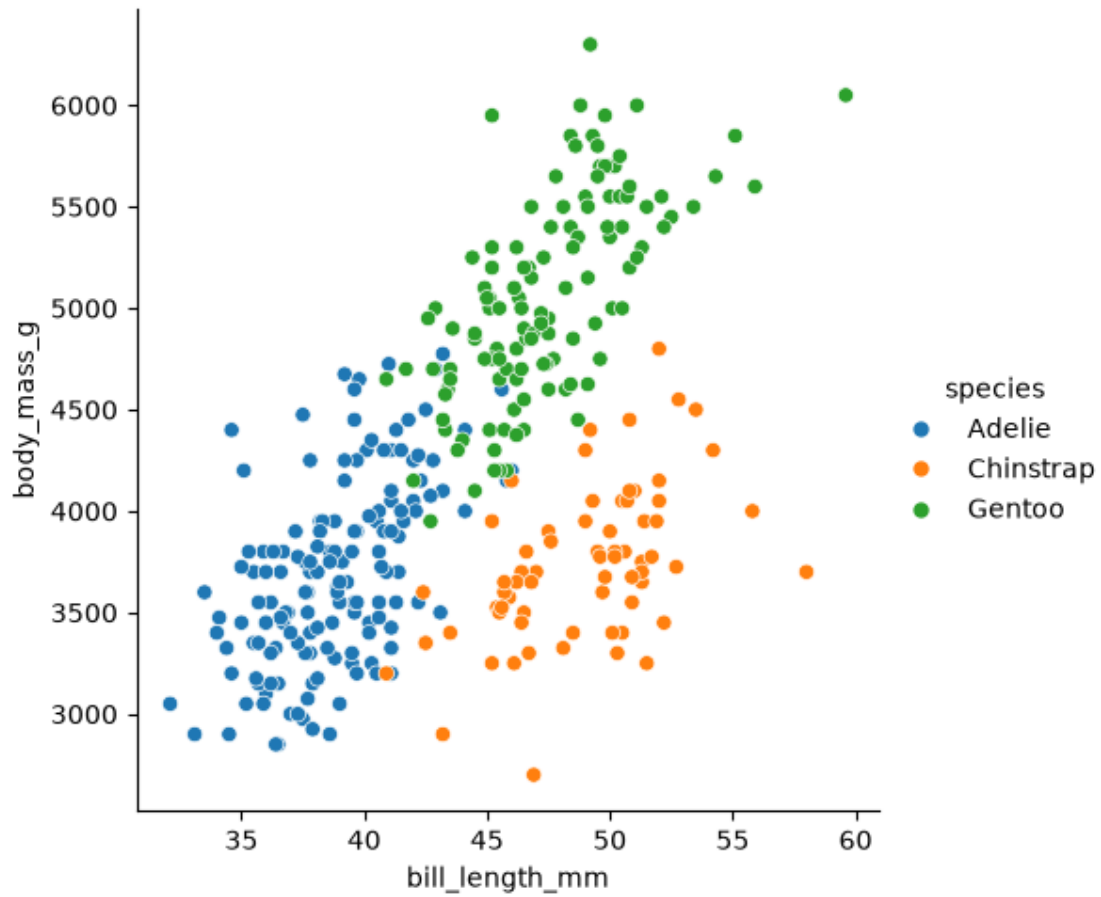
plt.show()
```



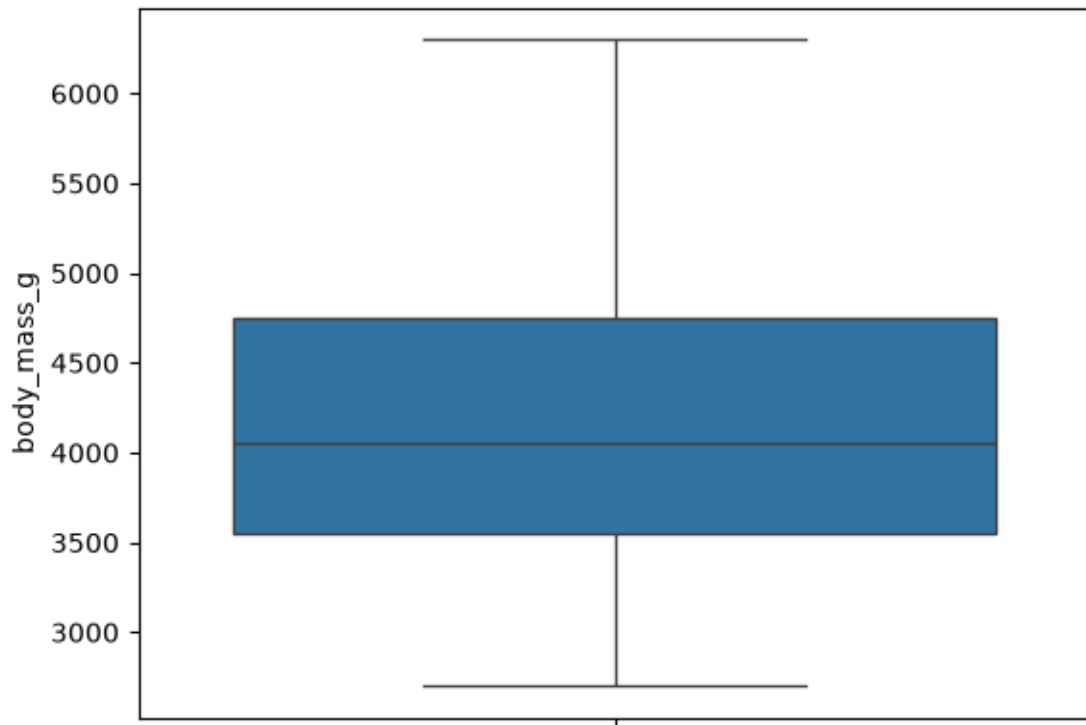
```
[58]: # =====
# Comparar dos variables utilizando una variable categórica.
# =====

sns.relplot(
    x="bill_length_mm",
    y="body_mass_g",
    hue="species",
    data=penguins,
    kind="scatter"
)

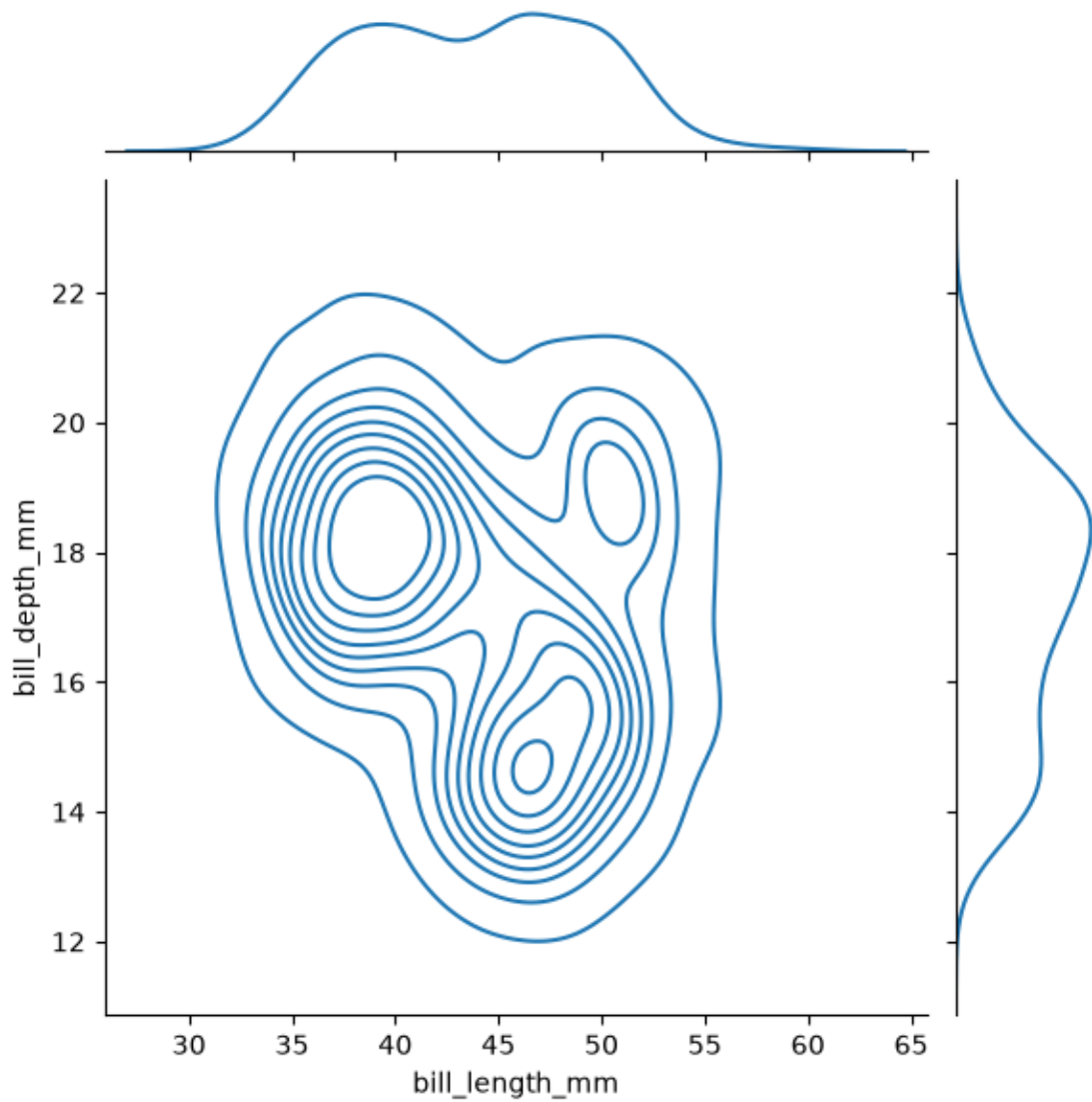
plt.show()
```



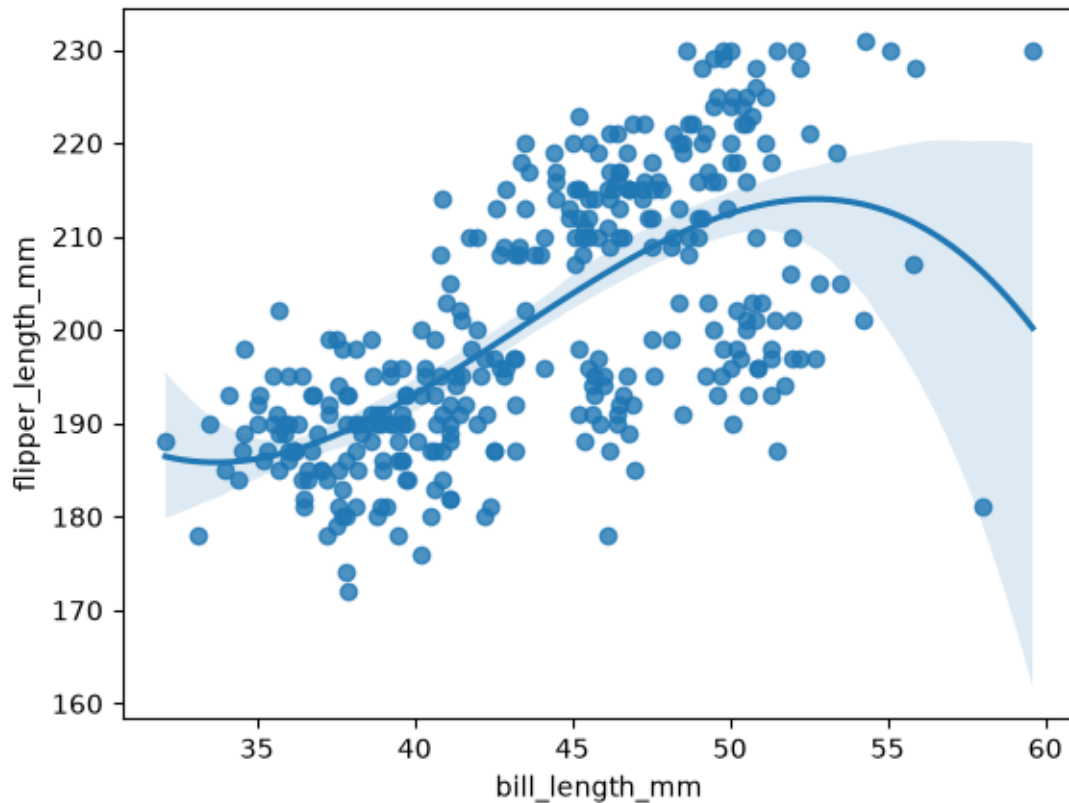
```
[59]: # =====  
# Distribución de una variable continua.  
# =====  
  
sns.boxplot(  
    y="body_mass_g",  
    data=penguins  
)  
  
plt.show()
```



```
[60]: # =====  
# Comparar la distribución de dos variables.  
# =====  
  
sns.jointplot(  
    x="bill_length_mm",  
    y="bill_depth_mm",  
    kind="kde",  
    data=penguins  
)  
  
plt.show()
```



```
[61]: # =====  
# Comparar una regresión lineal con una regresión polinómica.  
# =====  
  
sns.regplot(  
    x="bill_length_mm",  
    y="flipper_length_mm",  
    data=penguins,  
    order=3  
)  
  
plt.show()
```



9 Conclusión

Durante esta práctica se utilizaron las bibliotecas **Matplotlib**, **Pandas** y **Seaborn** para crear diferentes tipos de visualizaciones de datos. Se aprendió a representar información mediante gráficos de líneas, barras, áreas, histogramas, diagramas de dispersión y gráficos estadísticos, además de analizar distribuciones, correlaciones y relaciones entre variables. Estas herramientas constituyen una base fundamental para el análisis exploratorio de datos y para el desarrollo de proyectos relacionados con la ciencia de datos y el aprendizaje automático.

[]: